Apple Garage Website

Jiegao Chen



Figure 1: logo

©2025 Apple Garage Website

Contents

1. Analysis 4
1.1. The Problem 4
1.2. Research 5
1.3. Interviews 8
1.4. Further research and Prototyping 9
1.5. Objectives 13
1.6. GUI 16
1.7. Software Tool Selection 21
2. Documented Design 22
2.1. Structure diagrams 22
2.2. General UI 23
2.3. Landing page 25
2.4. Login 26
2.5. Sign Up 29
2.6. Customer landing page 31
2.7. Add New Car 32
2.8. Book services 36
2.9. View car detail 39
2.10. Vehicle Maintenance record 40
2.11. Staff landing page 41
2.12. Attendance 42
2.13. Working page 44
2.14. Add staff working with 45
2.15. View additional notes 46
2.16. Add car part 47
2.17. Add problem encountered 48
2.18. Managing job status 49
2.19. Manager page 50
2.20. Create staff account 52
2.21. Database design 53
2.22. File structure 56
3. Technical Solution 57
3.1. Python code 57
3.2. HTML code 73

©2025 Apple Garage Website

Layout.html 73
Landing.html 77
Login.html 78
SignUp.html 79
ResetPassword.html 80
Customer.html 82
AddCar.html 84
Error.html 85
Detail.html 86
Car_detail.html 88
Booking.html 89
Record.html 92
Staff.html 94
Additional_note.html 97
Job.html 98
Attendance.html 104
jobHistory.html 106
Manager.html 108
serviceDetail.html 111
CreateStaffAccount.html 113
4. Testing 115
5. Evaluation 116
5.1. Objectives 116
5.2. User feedback 120
5.3. Improvements 121
Appendix - Techniques used 122

©2025 Apple Garage Website

1. Analysis

1.1. The Problem

In today's fast-paced world, owning a car has become a necessity for many individuals, enabling convenience and mobility in daily life. To keep vehicles in good condition and ensure safety on the road, regular maintenance and timely repairs are essential. However, the process of finding a reliable car service provider and scheduling appointments can often be a challenging, time-consuming, and frustrating experience for car owners.

Currently, many car service businesses rely on traditional methods for managing appointments and storing service records. These methods typically involve phone calls, walk-ins, and manual record-keeping, which are not only inefficient but also prone to errors. While some businesses have taken steps toward modernization by adopting basic online booking systems, these systems often lack functionality and are outdated. Making them inefficient for both customers and service providers.

Recognizing these challenges, I decided to create a garage management website designed to enhance the operational efficiency of a garage and improve user experience for customers. This application will cater to three different types of users: customers, staff, and manager. The aim of this website is to create efficient and useful functionalities for each user group.

©2025 Apple Garage Website

1.2. Research

To able to gain more contents needed to include in my website. I researched a few exist websites. the common features they included are:

- Login/Sign-Up
- Booking Services – Online appointment scheduling for car repairs, maintenance, etc.
- Service History – View past services, invoices, and repair history for their vehicles.
- Vehicle Management – Add and manage multiple vehicles (with details like make, model, registration, etc.).

Following are functionalities particularly attract me which could be good also used in my website:

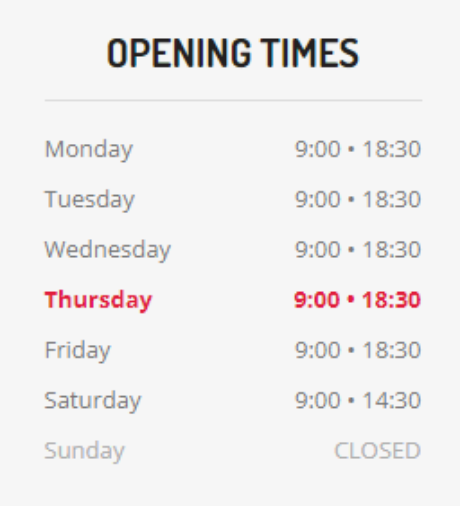


Figure 2: opening times

In figure 2 it shows clearly its opening time at its home page for users. Users able to see this information when they turned on this website. So, they can make decision immediately whether visit or not.

©2025 Apple Garage Website

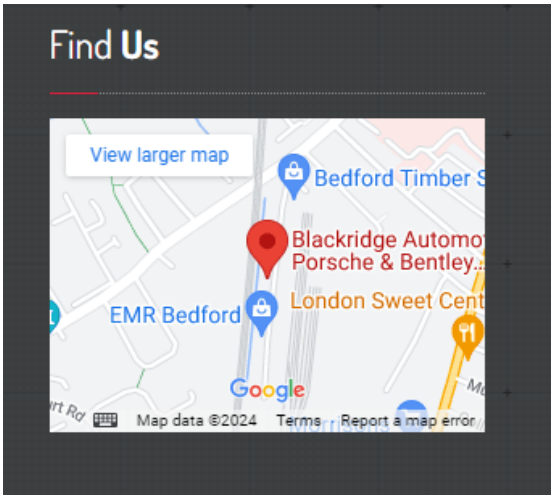


Figure 3: map

In figure 3 it shows the garage location on google maps. Making it easier for customer to find the garage location.

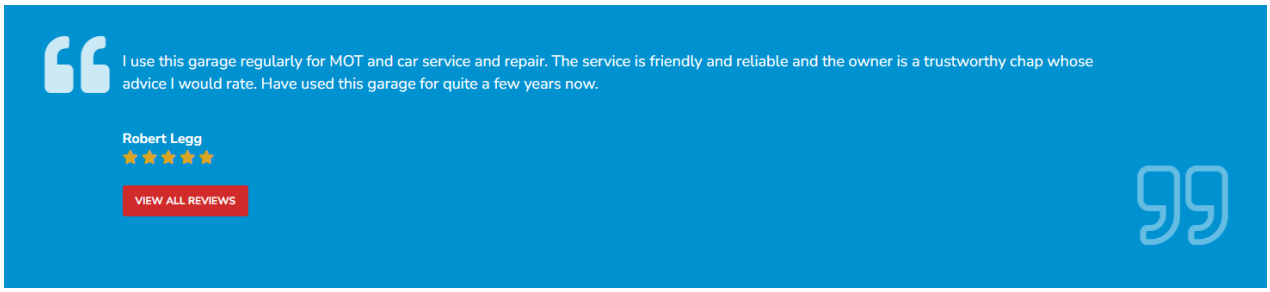


Figure 4: reviews

In figure 4 it shows the function allowing customers to comment and rate their services. They are also available for customers to read other customers reviews. This can make the service more transparent to attract customers.

©2025 Apple Garage Website

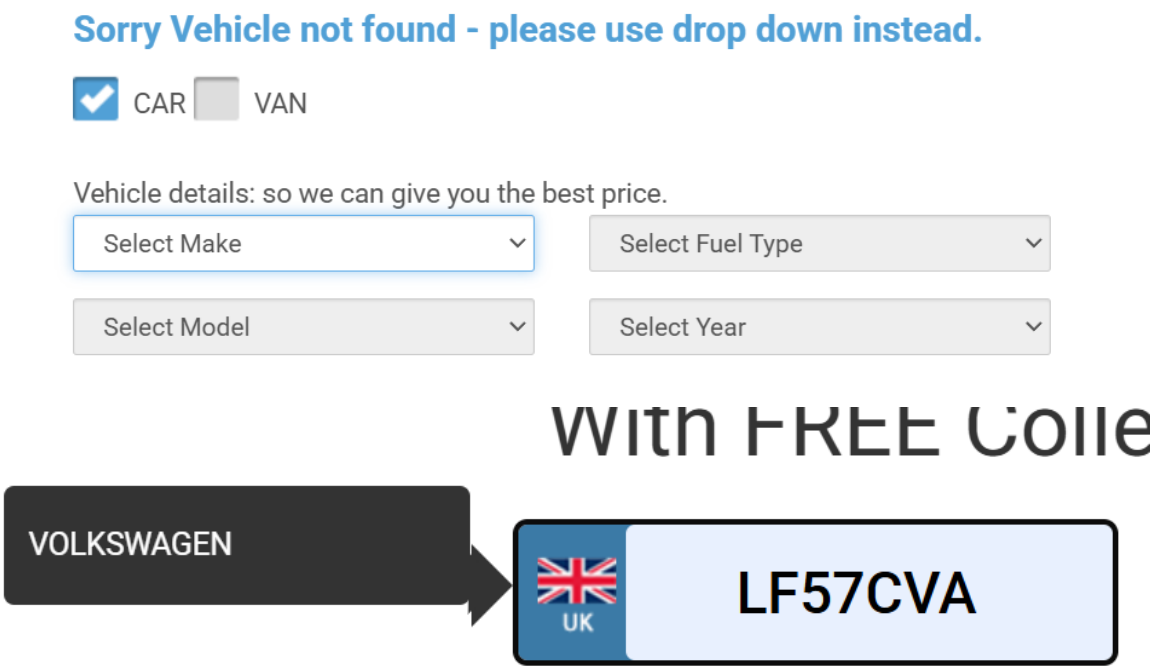


Figure 5: Identifying car brand

Figure 5 shows that this website allows users to retrieve their vehicle's make and other relevant details simply by entering their car registration number. If the system is unable to retrieve the information, users are given the option to manually input their vehicle details. Having this functionality would reduce the amount of data customer need to enter manually when registering their car. Additionally, it would enhance data accuracy for staff, as manual entry increases the risk of errors in vehicle details.

©2025 Apple Garage Website

1.3. Interviews

My father works in a car repair garage, so I decided to do an interview to my father to gain more information on how garage work and what its needs if I create a website for them.

- **Can you describe the garage?**

We currently have 10 employees and can repair eight vehicles at the same time. We perform MOT and repair engine, brake, suspension & steering, transmission, cooling & heating and electrics & lighting.

- **What is the garage operating time?**

9 hours per day Monday to Friday, from 9am to 18pm.

- **What are the most common services or repairs requested by customers?**

A full service which includes oil change, air filter change, fuel filter change, pollen filter change

- **How do customers currently contact the garage?**

Phone calls and walk-ins.

- **How do you keep track of customer information and vehicle history?**

Not stored at all, rely on employees' memory.

- **How do you track the progress of vehicle repairs?**

We currently don't have a system to track the progress of vehicle repairs.

- **How do you get car parts?**

We order our car parts on the website called Euro Car Parts.

- **How do you currently handle payments?**

Cash and card

- **What features would you like to see on a garage website?**

Features could store and access car part used in services and problems encountered during the services. Also have feature can access vehicle details more easily and quickly.

- **What features do you think would make a garage website useful for customers?**

An online booking system for customers with storage and retrieval of maintenance history.

©2025 Apple Garage Website

1.4. Further research and Prototyping

API

To enable easier and quicker access to vehicle details. I researched and found vehicle detail API could be useful to enable quick access vehicle details. After further research into existing vehicle APIs, I identified a free vehicle detail API on UK government website called Vehicle Enquiry Service (VES) API. It uses the vehicle registration number as input to search and provide details of the vehicle. The response data is provided in JSON format.

To learn how to use this API and understand its response. I investigated the code example they provided and created a prototype based on using this API. here's what I got:

```
url = https://driver-vehicle-licensing.api.gov.uk/vehicle-enquiry/v1/vehicles
```

This is the API endpoint that processes vehicle registration queries.

```
print(response.json())
```

I converted the API response from JSON format into a Python dictionary

- **API responses:**



Figure 6: API responses

©2025 Apple Garage Website

Figure 6 shows the responses of this API. By entering a vehicle registration number, I can retrieve multiple important details for maintenance about the vehicle, including its MOT status, make, weight, and year of manufacture. These results align with my objective of enabling quicker and easier access to vehicle details. So, I have decided to use this API into my entire program.

However, during testing, I also discovered that the API does not provide the car's model. To address this limitation, I plan to let customers to manually input their car model when registering their vehicle. This ensures the completeness of the vehicle information in the system.

Data storing

In preliminary research and planning, there a lot of data storage and retrieval is required to enable all the functions to working properly. For example: storing all services records and car maintenance history. So, I decided to use database and SQL to store, update and query data from database.

Entity-relationship diagram

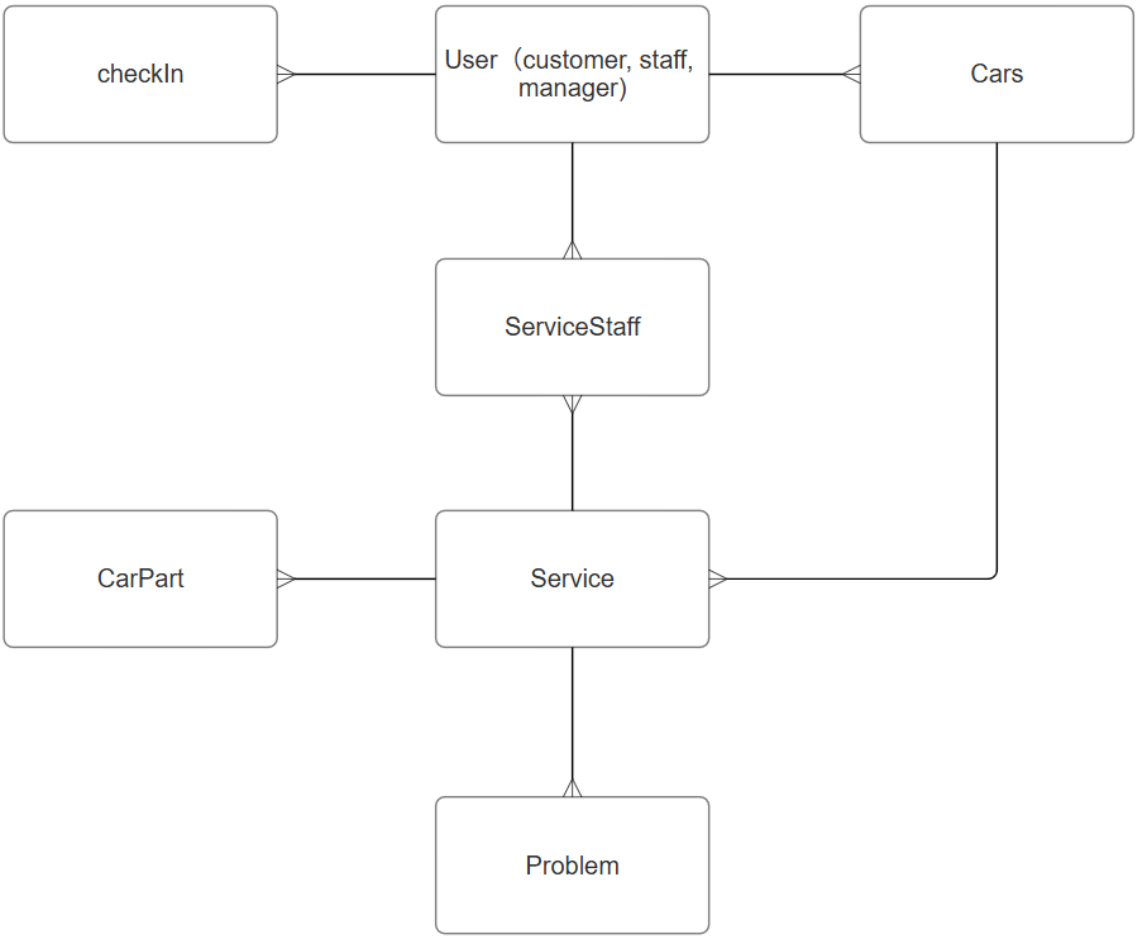


Figure 7: Entity-relationship diagram

©2025 Apple Garage Website

Tables

After drawing the ERD diagram. I identify need 7 tables to store all the data generated by the website. Here's my planned contents for each table:

- Table User – user details and user types.
- Table Car – vehicle details and car model entered by customer.
- Table Service – service records and their status.
- Table Parts – car parts used during the services
- Table Problem – problems encountered during the services
- Table CheckIn – staff's attendance.
- Table ServiceStaff – staff worked in which services

SQL

SQL is a standard programming language used to interact with and manage relational databases. Following are examples of some SQL language I going to using store, update and query data.

- SELECT – selects specific records form the table that meet certain criteria
- FROM – Identify which table the records will come from
- WHERE – adds the criteria that records returned need to meet
- DELETE – used to delete data from a database table
- INSERT INTO – used to add data to a database table
- JOIN – allows to select data from multiple tables at once
- UPDATE – Allows records to be updated

©2025 Apple Garage Website

Car parts

After discussing with my father following the interview, he suggested it would be beneficial to have a function for storing all car parts used in services. To explore how this could be done accurately and efficiently, I decided to research the website eurocarparts.com, where they purchase car parts.

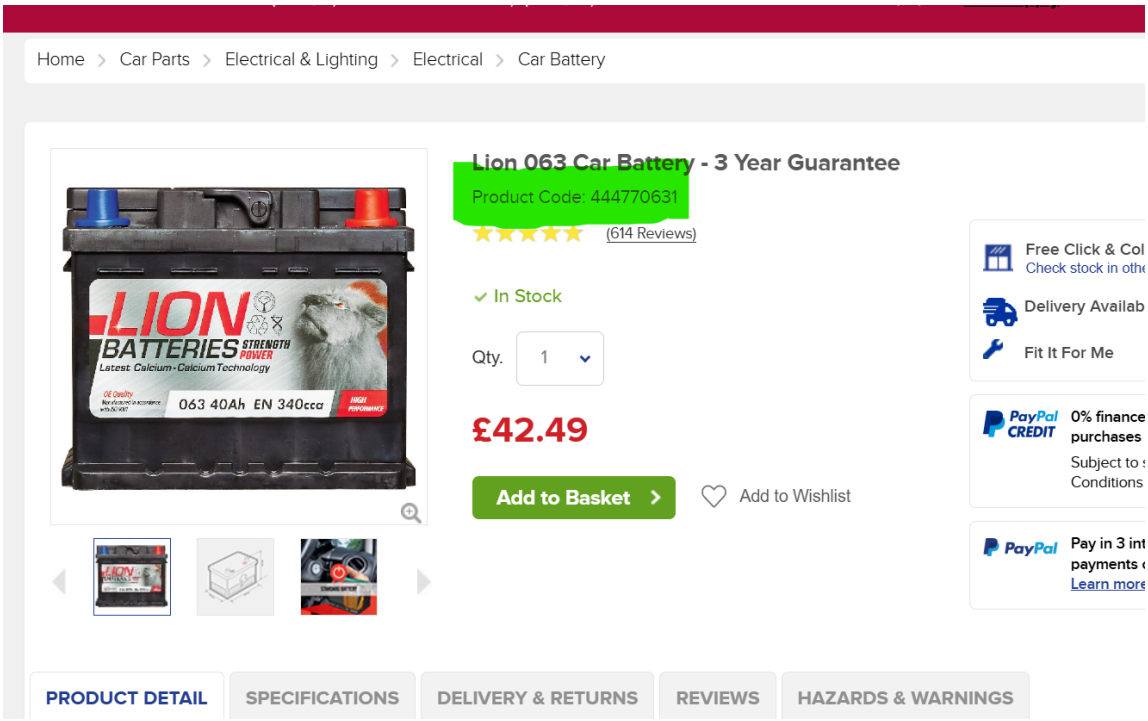


Figure 8: eurocarparts.com

During my research, I discovered that Euro Car Parts assigns a unique product code to each item they sell. This gave me the idea of enabling staff to input the product code of the car parts used during a service into the system, which would then be stored in the database. When needed, staff and manager could simply search for the product code on eurocarparts.com to quickly retrieve detailed information about the specific part used.

©2025 Apple Garage Website

1.5. Objectives

1. Home page and login system

    1.1. Landing page

        1.1.1. The logo

        1.1.2. The detail and explanation of the garage

    1.2. Login

        1.2.1. Allow users to enter their username and password to login

        1.2.2. Have button to jump to sign up page

        1.2.3. Have button to jump to reset password page

        1.2.4. Redirect to correct user page based on user type once login successfully

    1.3. Signup

        1.3.1. Allow users to enter their username and password to create account

        1.3.2. Redirect to login page once create account successfully

    1.4. Reset password

        1.4.1. let users to enter correct old password and then replace it with new password

2. Customer Functions

    2.1. My car

        2.1.1. list vehicles owned with registration number, make and model

        2.1.2. Customer able view details of the car

        2.1.3. Customer able to view maintenance records

        2.1.4. Have sook service button jumps to book service page

        2.1.5. delete car button

    2.2. Vehicle detail page, listing vehicle information from API

    2.3. Maintenance record page

        2.3.1. display to the user all maintenance records on their car including maintenance item, status, date and total expenditure

        2.3.2. allow user to cancel their booking if maintenance status is scheduled

    2.4. Book service

        2.4.1. allow user to select service, date and add additional note if need.

    2.5. Delete car, removing all the record on the vehicle

©2025 Apple Garage Website

2.6. Add new car

2.6.1. allow users to add their new car into the system by entering registration number

2.7. logout

3. staff functions

3.1. list of available jobs, including maintenance item, registration number, make, model and status

3.1.1. staff can start any scheduled jobs

3.1.2. staff can restart any paused job which paused by themselves

3.2. working page, page for staff during the maintenance period

3.2.1. staff can add staff they are working together with

3.2.2. staff can view details of the vehicle they are working with

3.2.3. staff can view additional notes by customer

3.2.4. staff can add car parts they used during the service by enter its name and product code

3.2.5. staff can view product code of car parts used history of the vehicle they are working with

3.2.6. staff can add problem they encountered during the service

3.2.7. staff can view problem history of the vehicle model they are working with

3.2.8. staff can pause job

3.2.9. staff can complete job

3.3. Attendance Page, staff can see their attendance for last 30 days.

3.4. Job history page

3.4.1. staff can see the table of jobs they did which including maintenance items, current status, maintenance date, car make, car model and registration number.

3.5. logout

4. manager functions

4.1. service record table, manager can see all jobs did by staffs

4.1.1. table including recordId, car make, car model, maintenance item, status, maintenance date, leader staff, assist staff, total expenditure.

4.1.2. record in table should list in order from new date to old.

©2025 Apple Garage Website

4.1.3. use different colours to represent records in different status in the table.

4.2. manager able to cancel scheduled bookings

4.3. manager able to enter total charge for service when its status is Done

4.4. manager able to create staff account

4.4.1. allow user to enter username and password to create staff account

4.5. logout

5. nav bar

5.1. nav bar will on top of every page with different options based on the user type

5.1.1. if user haven't login nav bar will display home, login and signup

5.1.2. if user type is customer nav bar will display my car, add new car and logout

5.1.3. if user type is staff nav bar will display home, attendance, job history and logout

5.1.4. if user type is manager nav bar will display records, create staff, logout

5.2. nav bar will also display username if user logged in

©2025 Apple Garage Website

1.6. GUI

Nav bar



Figure 41: nav bar GUI

Nav bar will appear at the top of every page. Logo and username will be displayed on the left of nav bar. Buttons for user quick jump between functions will be displayed on the right. Buttons will be displayed differently based on different user type.

Landing page

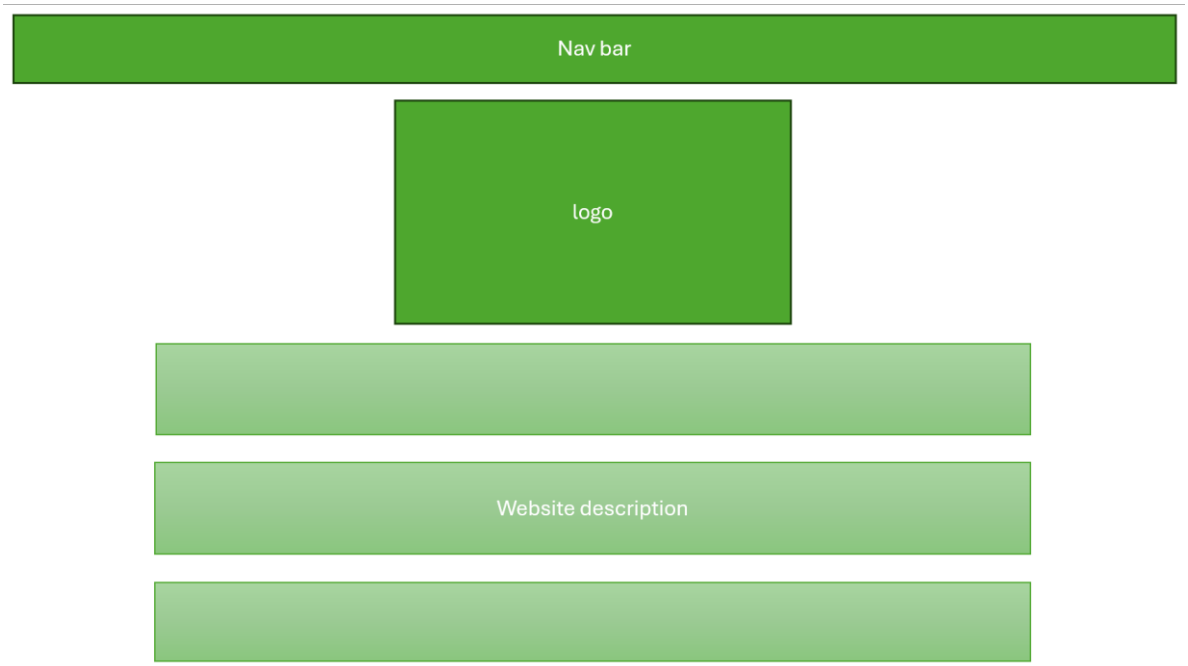


Figure 42: landing page GUI

For landing page nav bar will be displayed at the top. User can quick jump to login and sign up by clicking buttons on the nav bar. Logo and website descriptions will be displayed on the landing page.

©2025 Apple Garage Website

Customer and staff landing page

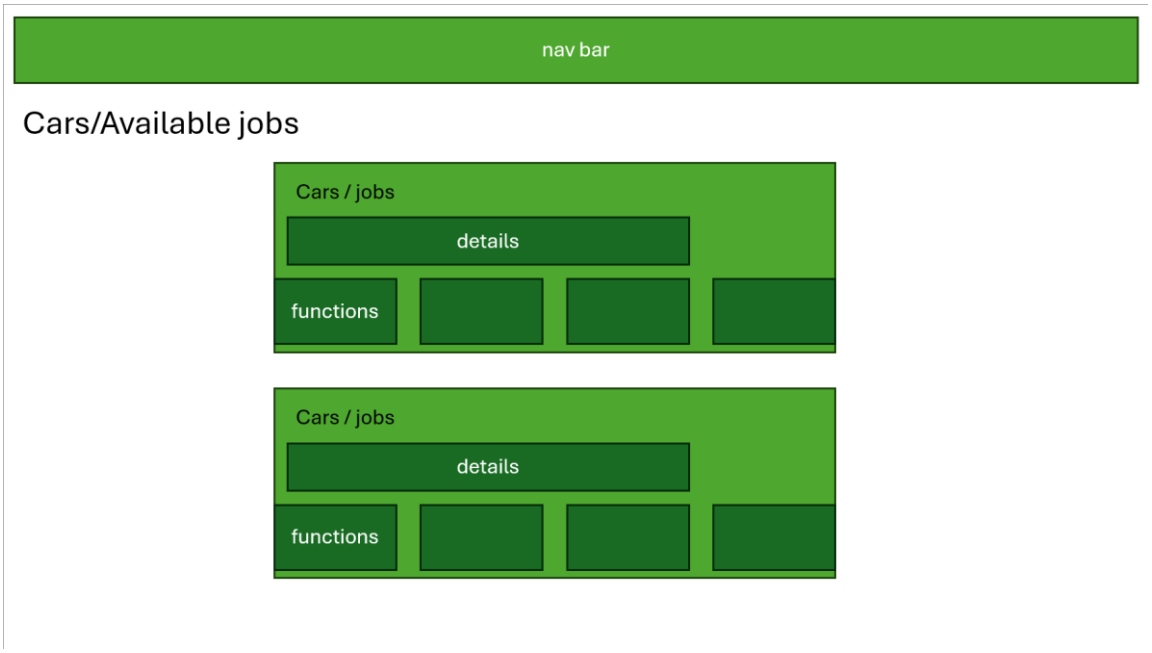


Figure 42: customer and staff landing page GUI

On the customer and staff landing will display all cars/jobs. Each block represents one car/job. User can perform actions they want on each car/job by clicking corresponding buttons in its block.

Booking service

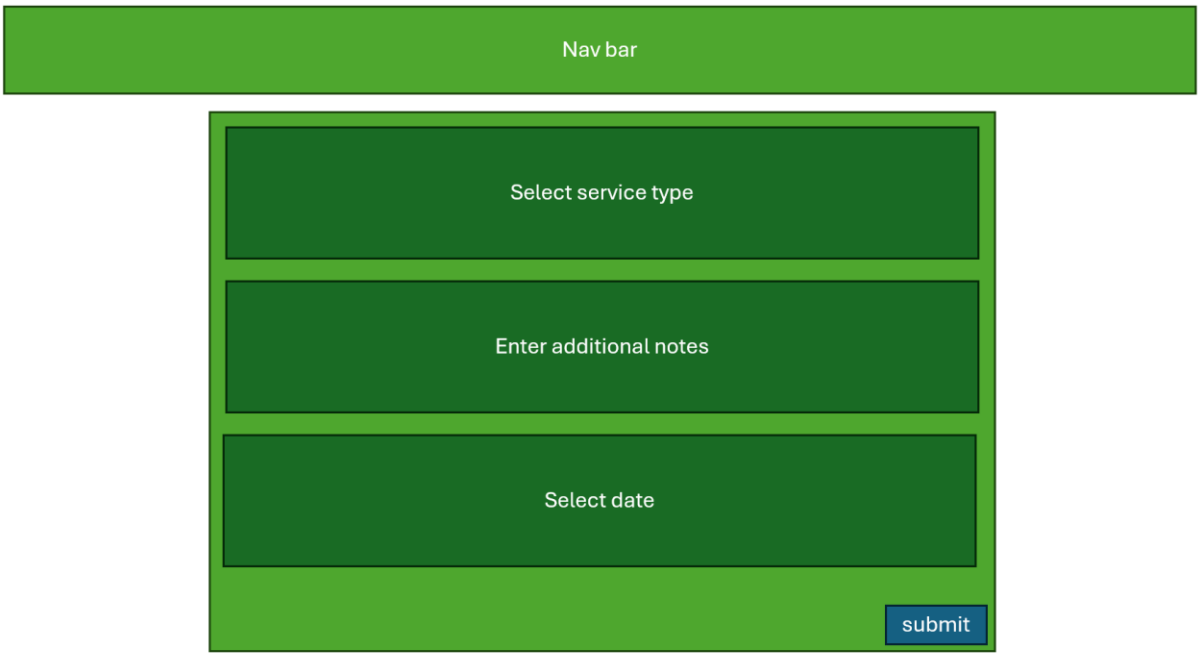


Figure 44: booking service GUI

©2025 Apple Garage Website

For customers to booking service on their car. They need to select the car they want to book on customer landing page. Then, select service type, date and enter additional notes (if have) to complete the booking.

Add new car

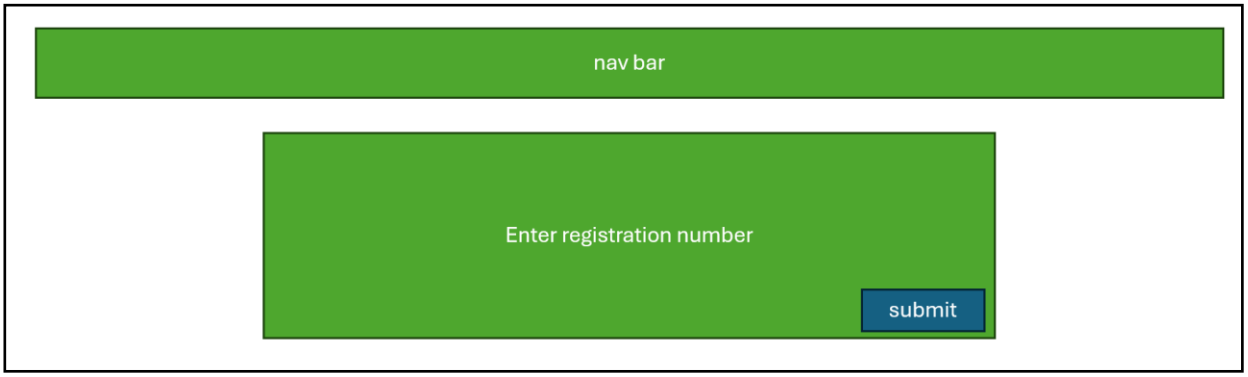


Figure 46: enter registration number GUI

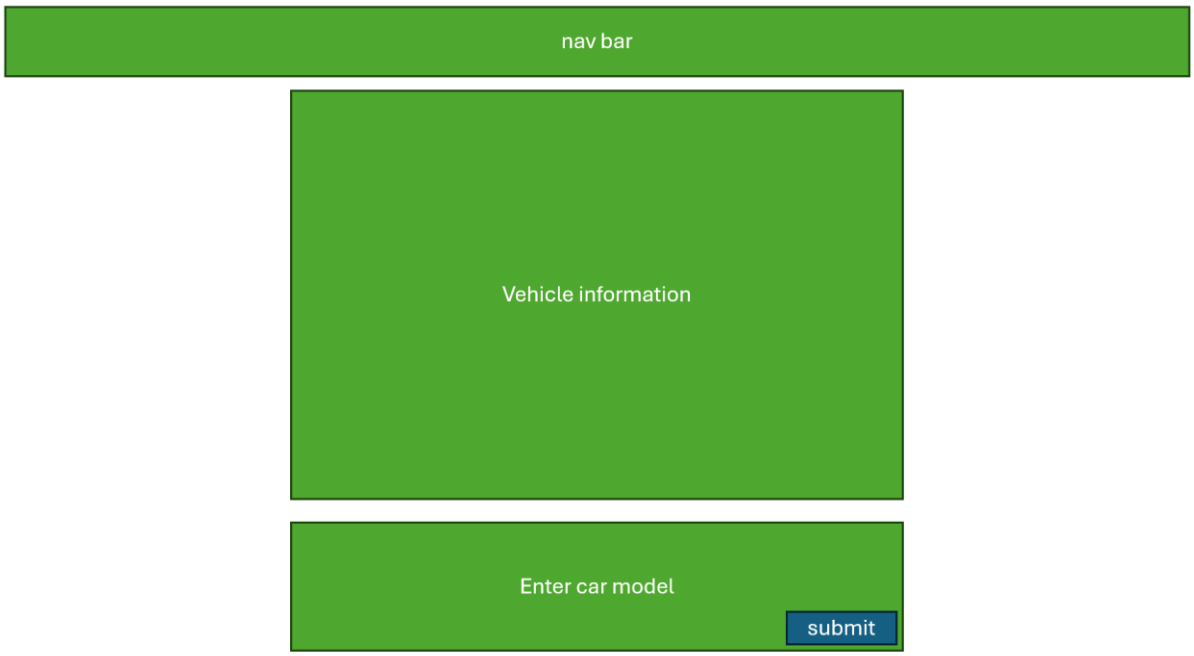


Figure 43: add new car GUI

For customers to add vehicle they had. They need to click add new car button on the nav bar on customer landing page. Once clicked will jump to page let user enter their registration number. when registration number entered will jump to page display their vehicle information get from DVLA API. Once customer confirmed their vehicle information and entered the car model, their car will be saved into database.

©2025 Apple Garage Website

Working page

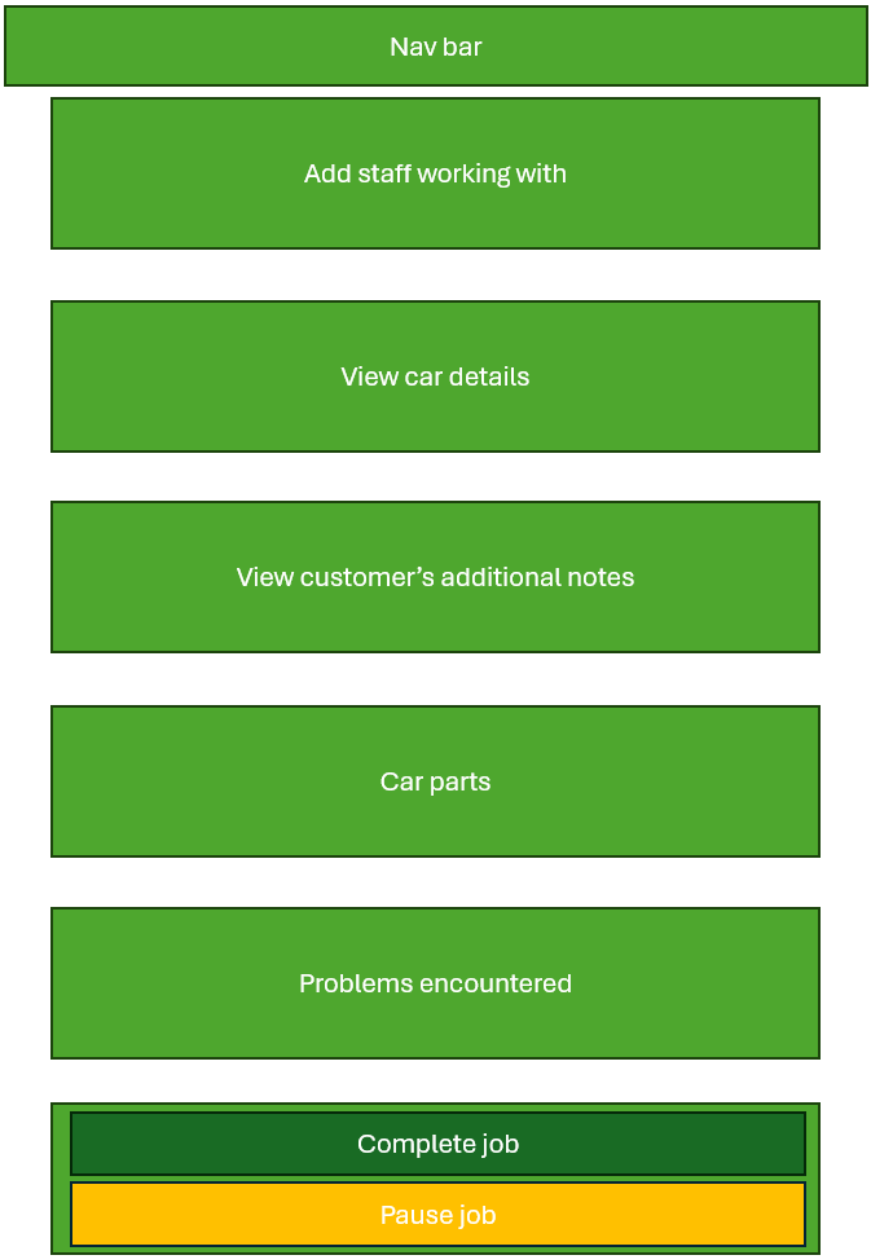


Figure 44: working page GUI

The staff working page is designed as a single continuous page to simplify operations. It is divided into small sections, with each section representing a specific function to enabling staff to quickly navigate between them.

©2025 Apple Garage Website

Manager page

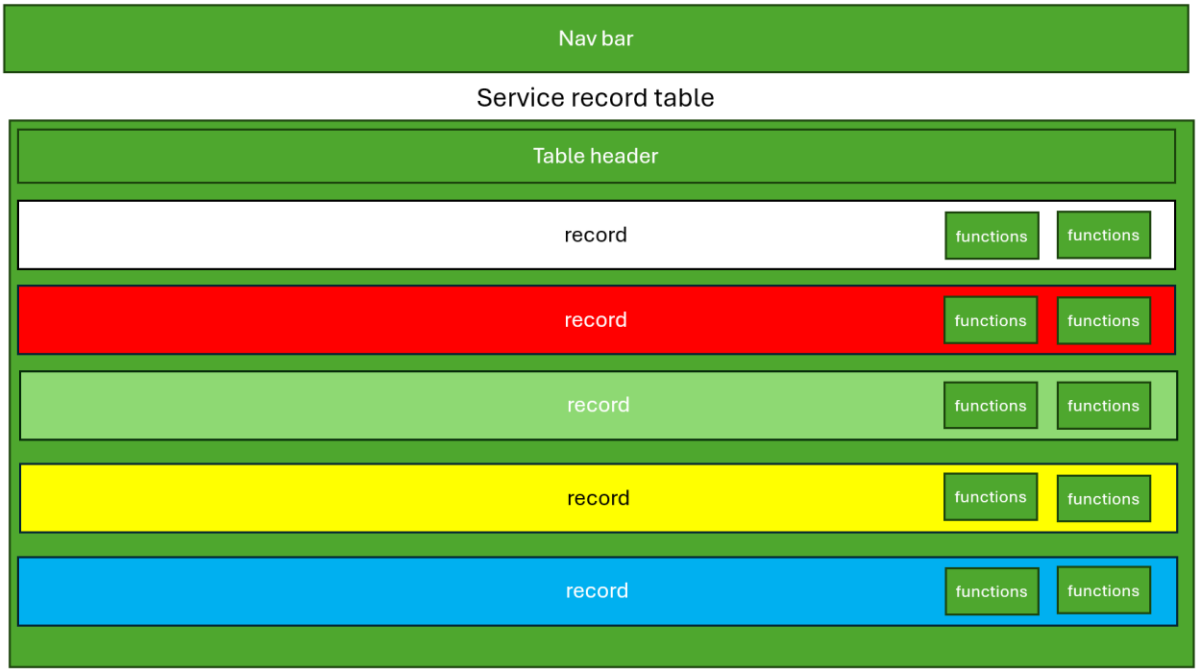


Figure 45: manager page

- The manager page will be displayed as a big table which contains all service records sorted from newest date to oldest. Each record will be displayed in different colour based on its status. (Red – paused, Green – done, Yellow – working, Blue – scheduled, White – paid). Functions can perform will be displayed on the right of each record.

©2025 Apple Garage Website

1.7. Software Tool Selection

 visual studio code – software to write all the code.

 DB browser for SQLite – database engine to running the database.

 Flask – building web applications, handling HTTP requests and rendering templates.

Requests library – make HTTP requests.

Sqlite3 library – allows to work with SQLite database.

Socket module – create network server.

Datetime module – manipulating dates and times.

Bcrypt – hashing passwords securely.

DVLA API – retrieve vehicle information

©2025 Apple Garage Website

2. Documented Design

2.1. Structure diagrams

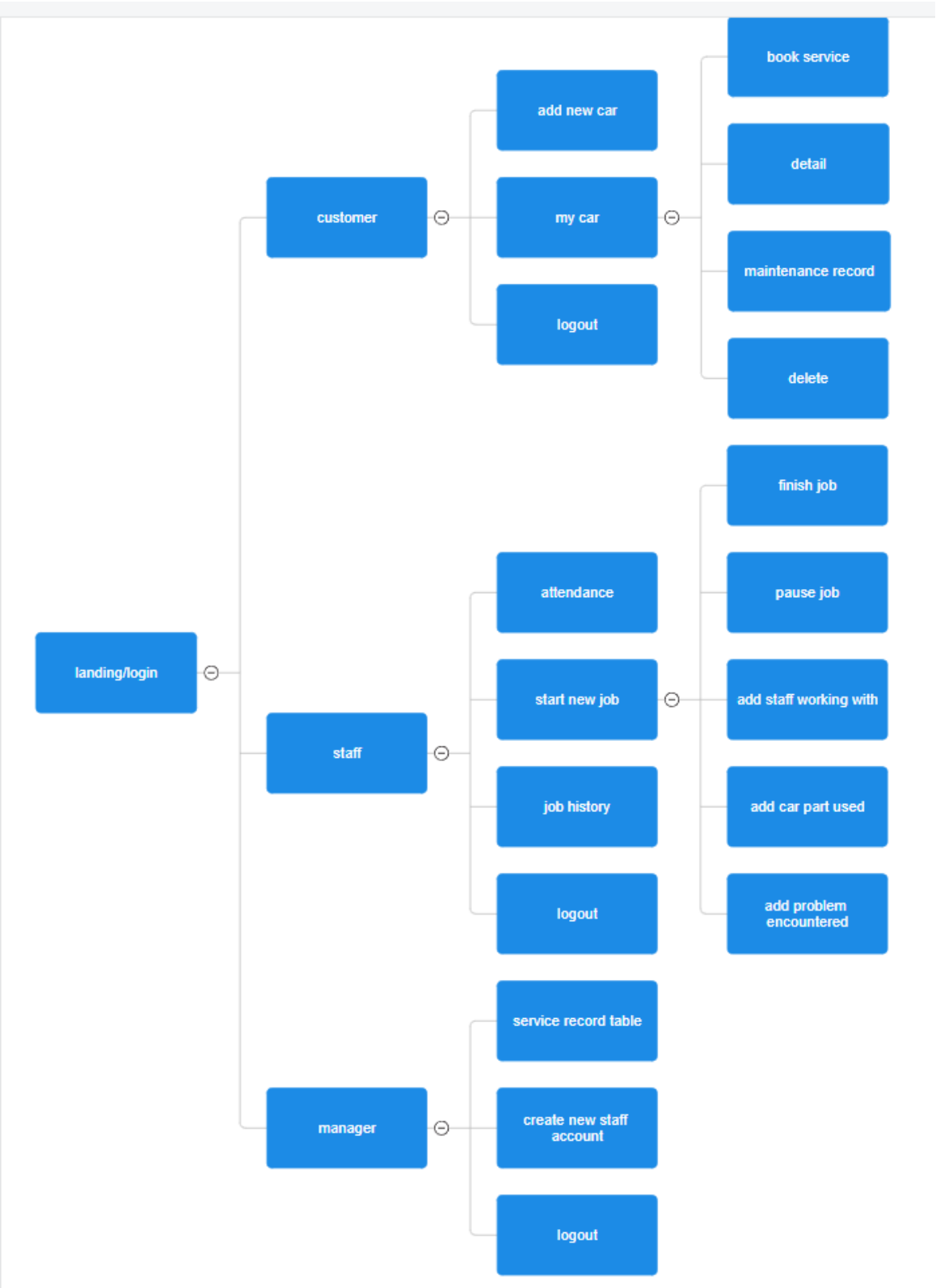


Figure 9：structure diagram

©2025 Apple Garage Website

2.2. General UI

The general UI of website will include a nav bar at the top, a footer at the bottom and a consistent background image for every page.

Nav bar



Figure 10: nav bar

Website name and logo

Buttons allow user jump between pages. This will changes depend on user type

Description

The nav bar will appear at the top of every page of website. The buttons on the right of nav bar will change as user type change. If user haven't login nav bar will display: Home, Login and Signup. If user type is customer nav bar will display: My car, Add new car and Logout. If user type is staff will display: Jobs, Attendance, Job history and Logout. If user type is manager will display: Records, Create staff and Logout.

Footer

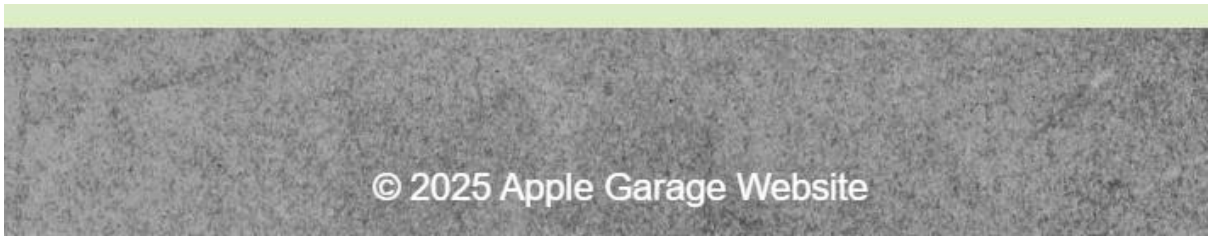


Figure 11: footer

Description

The Footer displays copyright of the website at the bottom of every page.

Colour theme

#5cba00

This colour is the theme colour to decorate pages on the website. To choose this colour because it matches the website logo, and green colour can create a sense of balance which helps users feel more relaxed and comfortable while browsing the site.

©2025 Apple Garage Website

Background image



Figure 12: background image

Fully view of general UI

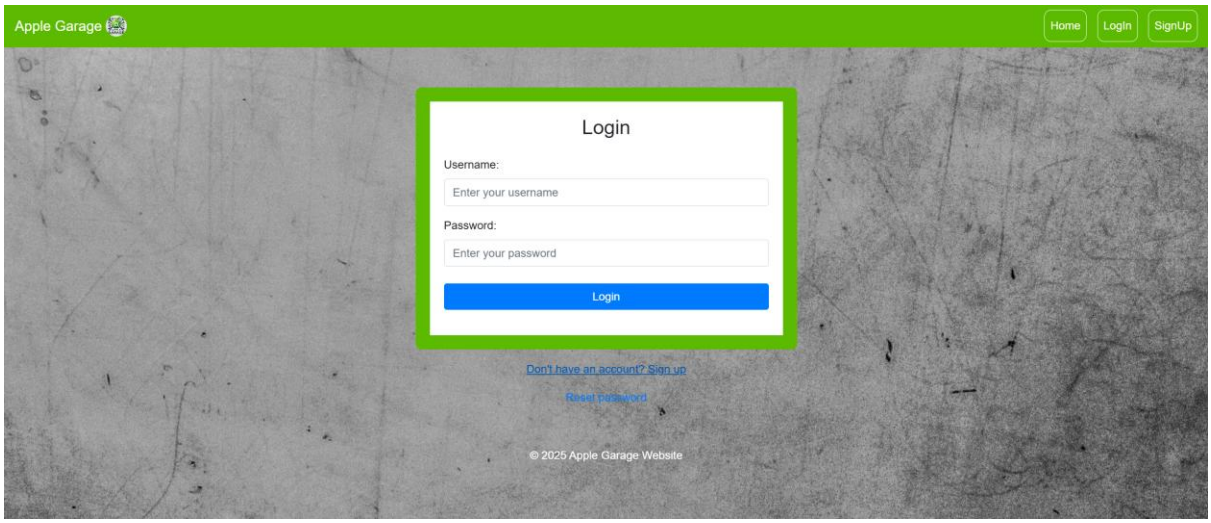


Figure 13： Fully view of general UI

©2025 Apple Garage Website

2.3. Landing page

Screen design

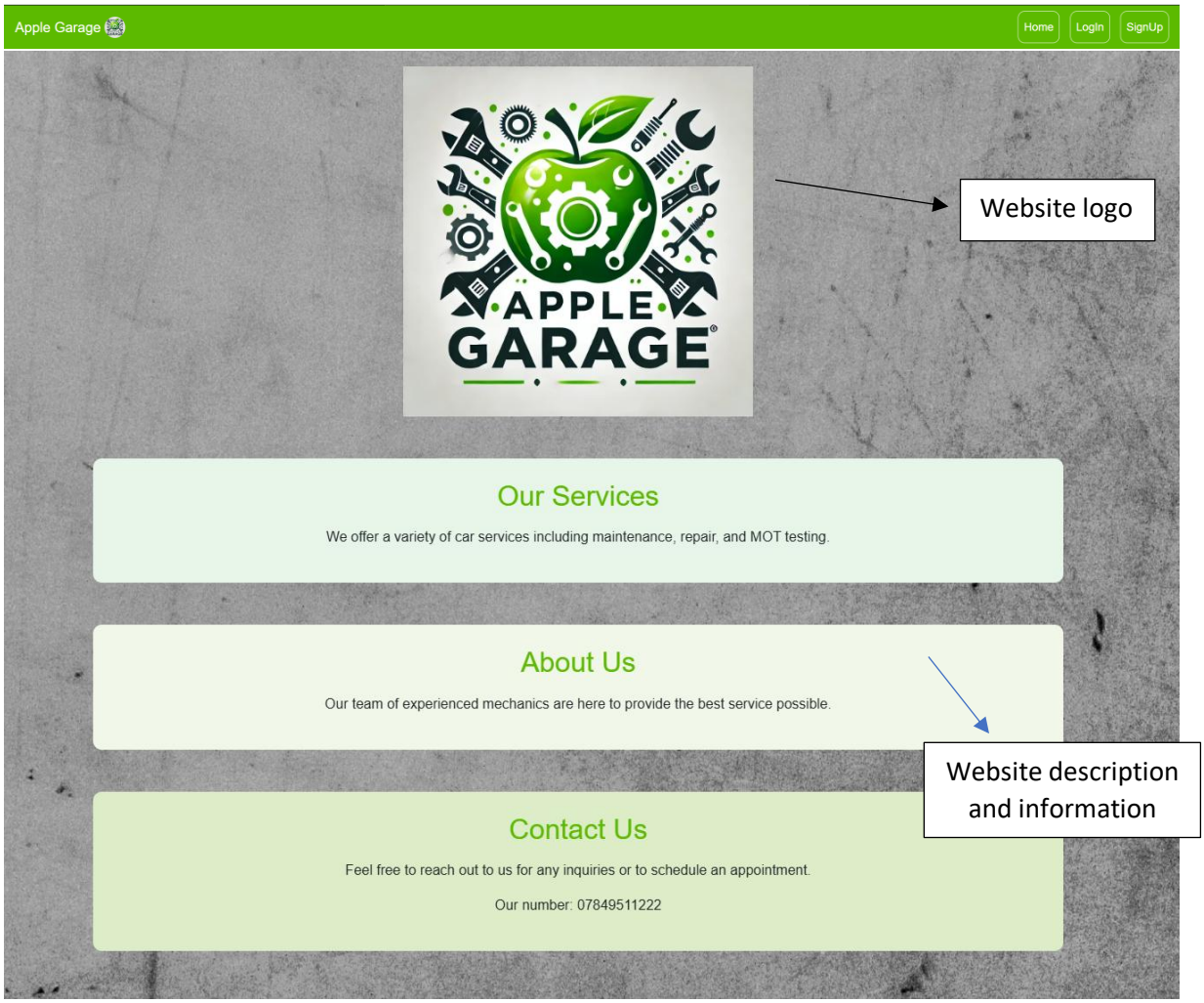


Figure 14: landing page design

Route

/

Description

This is the landing page design of the website. It includes two parts the logo and information of the garage. This will be the first page to displayed for every user when they turn on the website.

©2025 Apple Garage Website

2.4. Login

Screen design

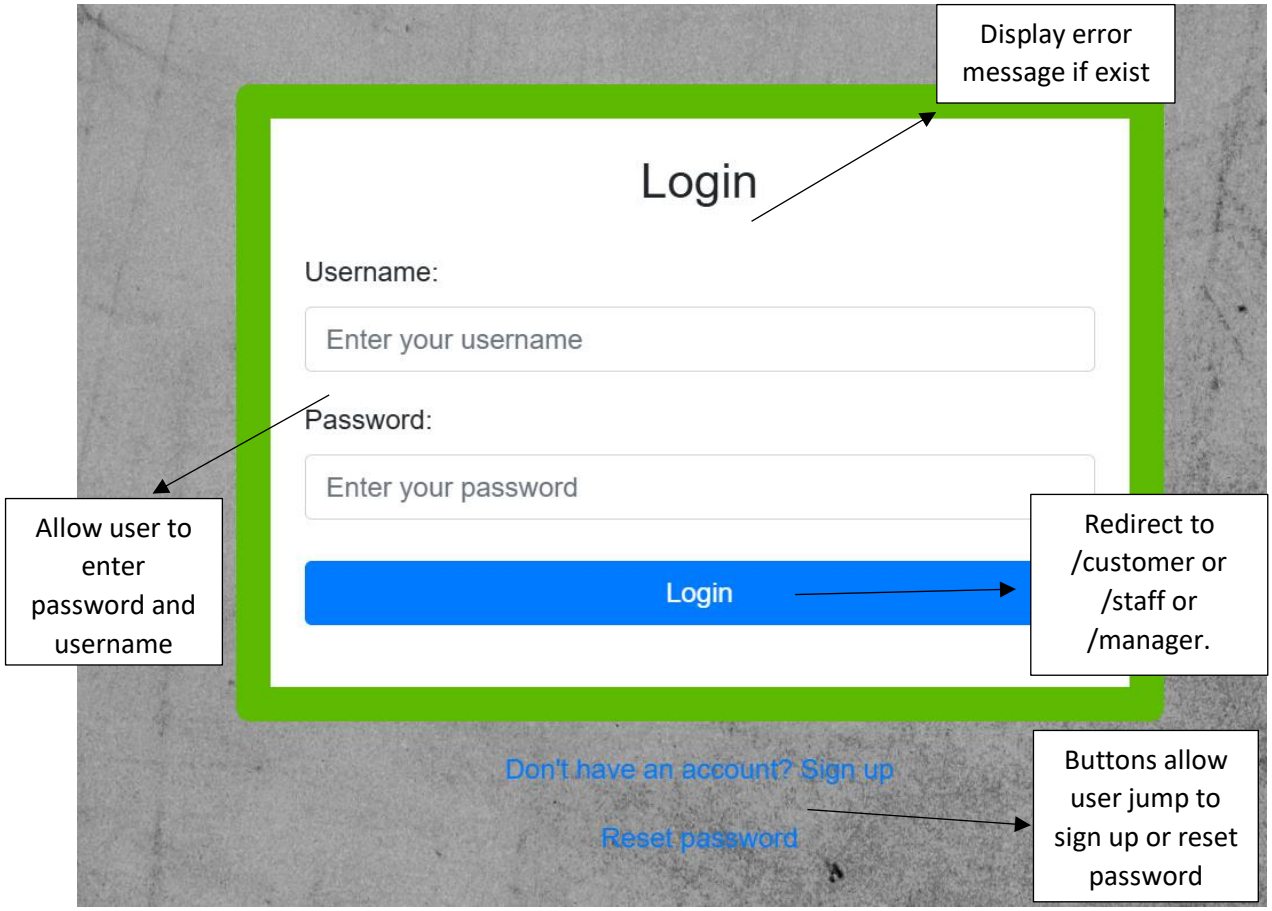


Figure 15: Login screen design

Route

/login

Description

This is login screen design. In this page users can enter their username and password. Once completed user can press button to process login. Algorithm will detect whether user input is valid or not and then detect which user type is this user belong to and jump to correspond page. Error message will only display when user is not valid to login (E.g.: wrong password, username does not exist).

©2025 Apple Garage Website

System flowchart

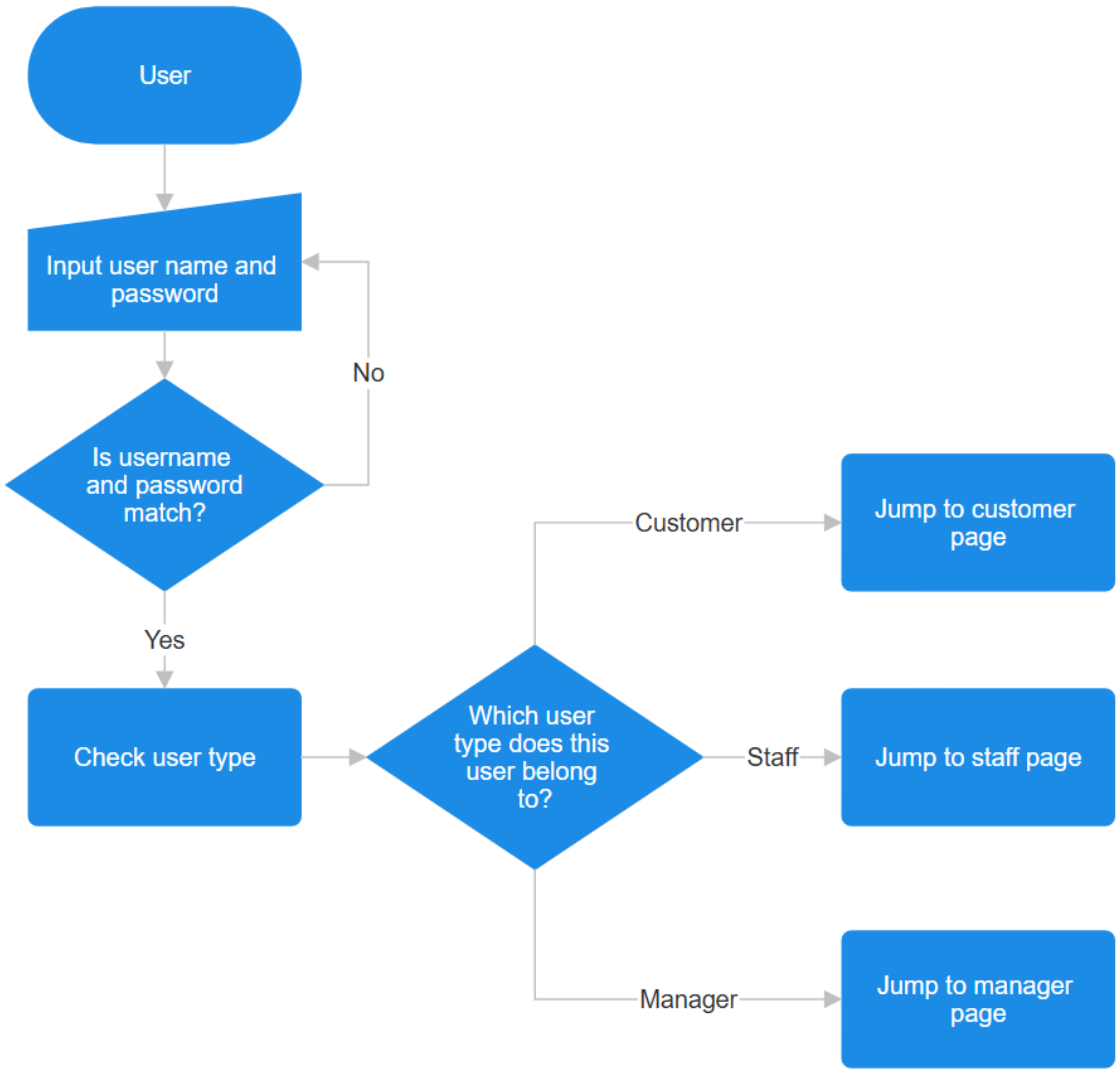


Figure 16: login system flowchart

©2025 Apple Garage Website

The following pseudocode first verifies whether the username exists and if the provided password matches it. Then, it determines the user type and redirects the user to the corresponding page.

Algorithm design – validation checking and detect user type

```
username = Get input from form field 'username'
password = Get input from form field 'password' (encoded for security)
Query the database to retrieve user details based on the provided username
Execute SQL query:
    SELECT userID, userPassword, userType
    FROM tblUser
    WHERE userName matches the input username
authenticate_data = Fetch the result of the query
IF authenticate_data exists AND Password matches the stored hashed password:
    Save userID in session['userId']
    Save username in session['username']
    user_type = userType from the authentication data
    Close database connection
    IF user_type is "staff":
        Redirect to '/checkIn' page
    ELSE IF user_type is "customer":
        Redirect to '/customer' page
    ELSE:
        Redirect to '/manager' page
ELSE:
    Close database connection
    Render the 'login.html' template with an error message:
        "Username or password is wrong. Please try again!"
```


©2025 Apple Garage Website

2.5. Sign Up

Screen design

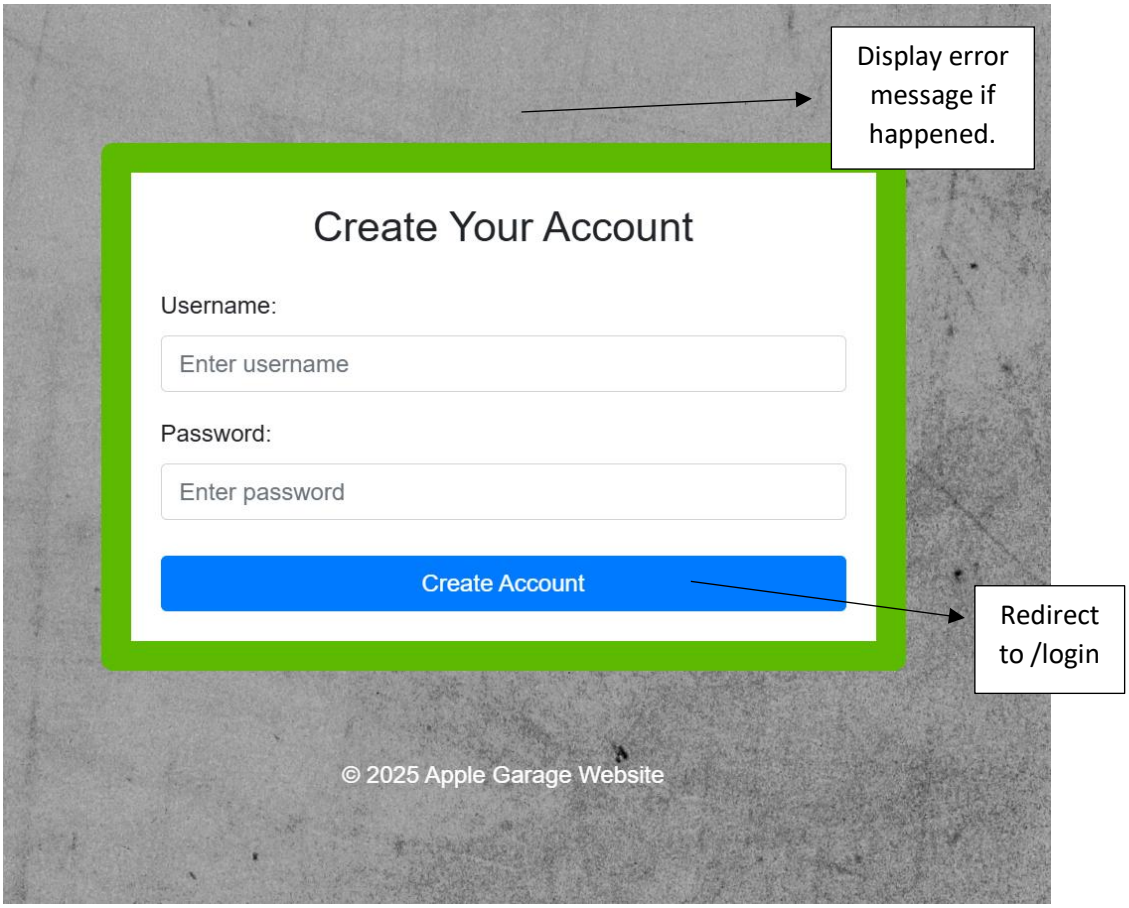


Figure 17: sign up page design

Route

/signUp

Description

This page allows users to create their account by entering the username and password they want. Once creation successful will jump to login page. The system will automatically detect if the username is duplicated. If happens will sent error message and let user create different username.

©2025 Apple Garage Website

System flowchart

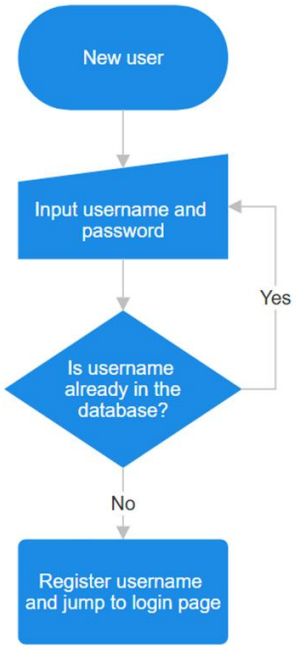


Figure 18: sign up flowchart

The following pseudo code is to get the username and password. It will then insert into database if username is unique

Algorithm design – create account and check duplicate username

```
username = get “username” field from request
raw_password = get “password” field from request
password_bytes = convert raw_password to UTF-8 bytes
salt = generate new bcrypt salt
hashed_password = hash password_bytes with salt using bcrypt
try
    execute SQL:
        INSERT INTO tblUser (id, username, password_hash, role)
        VALUES (NULL, username, hashed_password, “customer”)
    commit the transaction
    redirect to “welcome” page
catch any database error
    rollback the transaction
    render “signUp.html” with error message:
        “Username already exists. Please choose a different username.”
```


©2025 Apple Garage Website

2.6. Customer landing page

Screen design

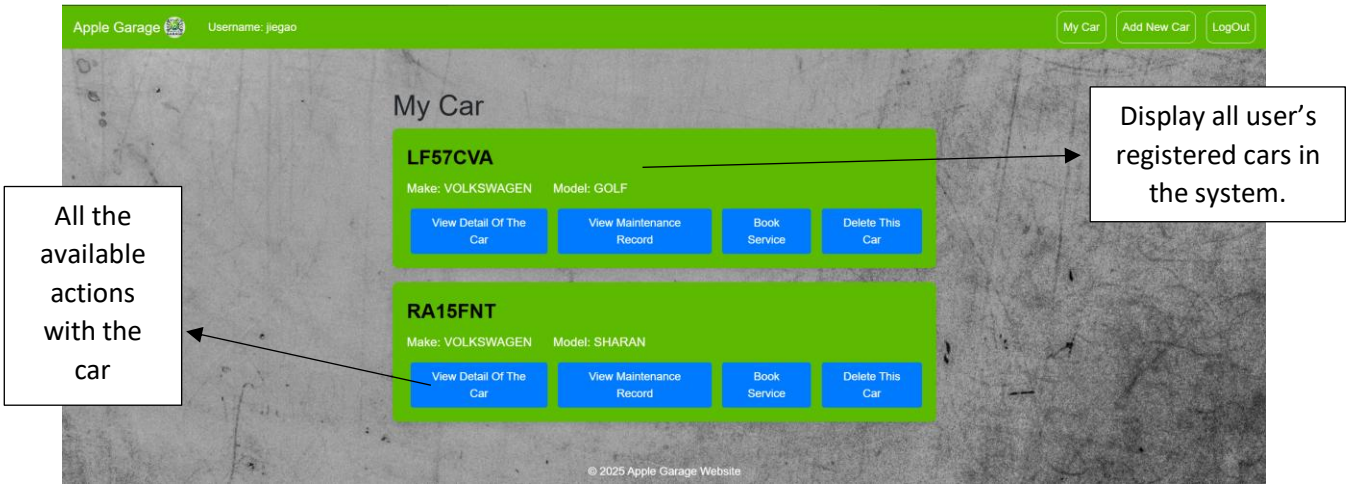


Figure 19: customer landing page design

Route

/customer

Description

This is landing page for customers. On this page will display cars user entered into system. User can implement actions they want with their car by clicking buttons underneath.

©2025 Apple Garage Website

2.7. Add New Car

Screen design – enter registration

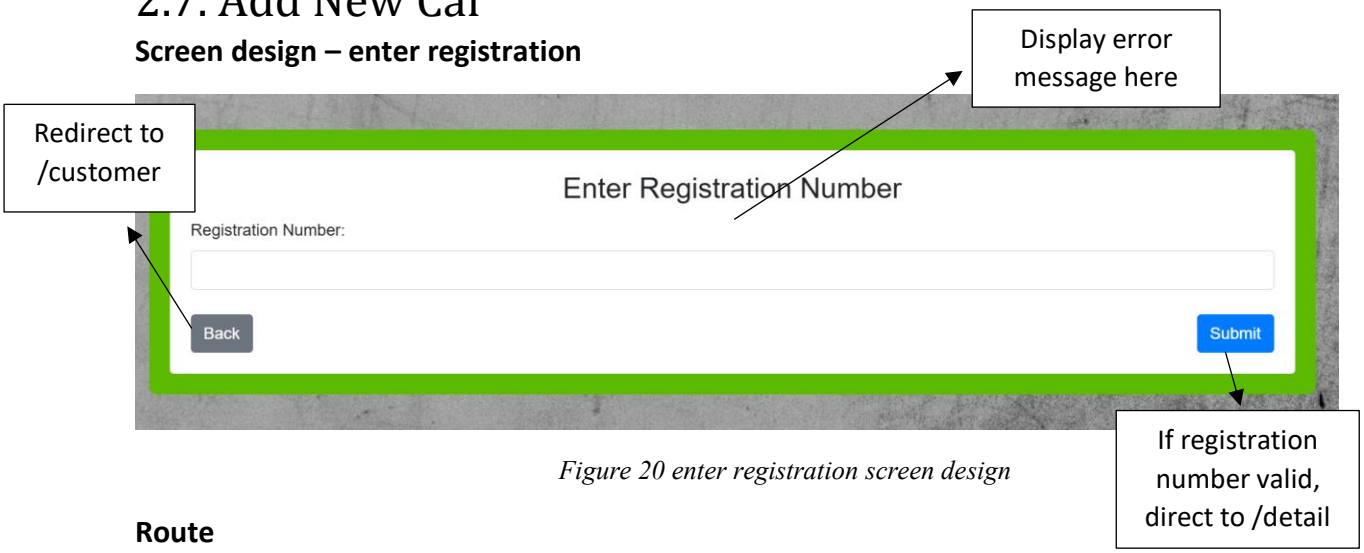


Figure 20 enter registration screen design

Route

/addCar

Screen design – display car detail

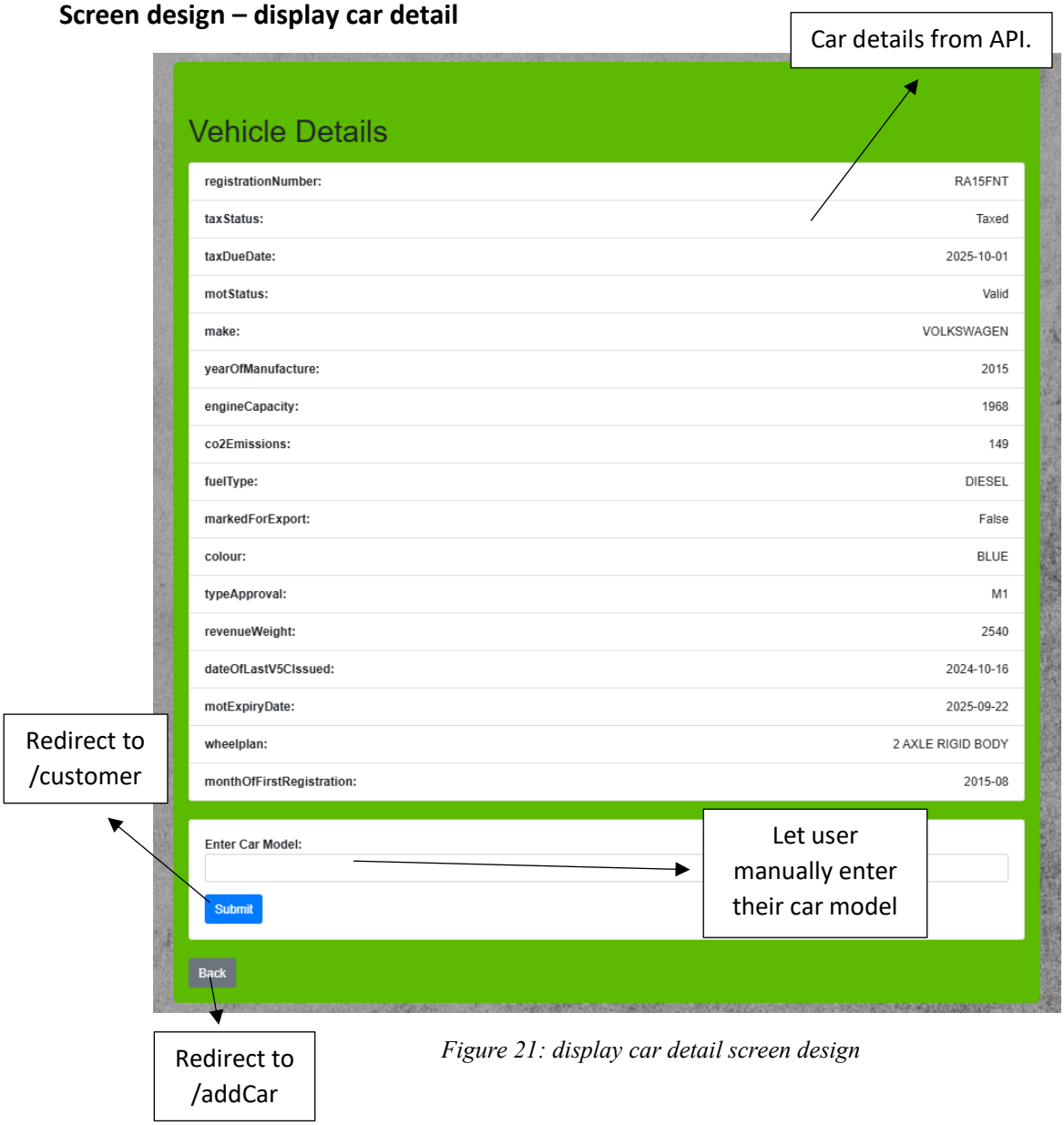


Figure 21: display car detail screen design

©2025 Apple Garage Website

Route

/detail

Description

Before users can book a service for their vehicle, they must first add their vehicle to the system. The process follows these steps:

1. Vehicle Registration Input: Users begin by entering their vehicle's registration number into the system.
2. API Verification: The system communicates with an external API to validate whether the registration number exists.
3. Duplicate Check: The system verifies whether the vehicle has already been added to prevent duplicate entries.
4. Vehicle Details Confirmation: If the registration number is valid and not already in the system, the system retrieves and displays the vehicle details for the user to confirm.
5. Manual Car Model Entry: Since the API does not provide the car model, users must manually enter this information.
6. Final Confirmation and Storage: Once all details are confirmed, users can save their vehicle in the system. The system will then store the information into database automatically.

System flowchart

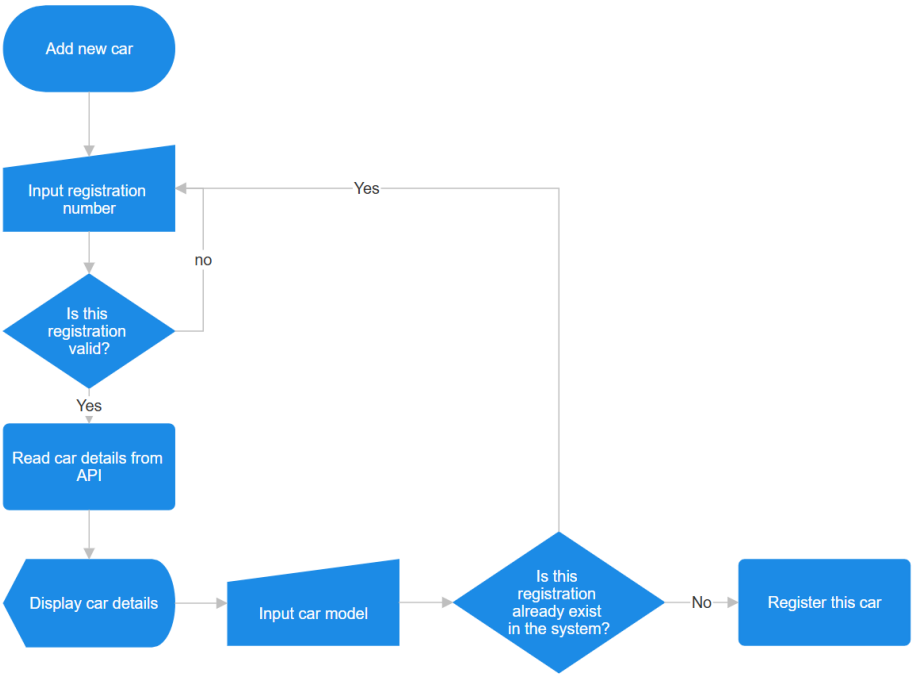


Figure 22: add new car system flowchart

©2025 Apple Garage Website

The following pseudo code is to retrieve vehicle information from DVLA API and convert them into python list.

Algorithm design – Retrieve vehicle details from API

```
DEFINE url AS "https://driver-vehicle-licensing.api.gov.uk/vehicle-enquiry/v1/vehicles"

SET reg TO user_input("Enter Registration Number")

SET payload TO {"registrationNumber": reg}

SET headers TO {

        "x-api-key": "your_api_key",

        "Content-Type": "application/xml"

    }

SET response TO SEND_POST_REQUEST(url, headers, payload)

SET title TO EMPTY_LIST

SET detail TO EMPTY_LIST

FOR EACH key IN response.JSON_KEYS():

    APPEND key TO title

FOR EACH value IN response.JSON_VALUES():

    APPEND value TO detail
```

The following pseudo code is to store customer's vehicle detail into database

Algorithm design – store vehicle details into database

```
    CONNECT TO DATABASE "carService.db"

    CREATE CURSOR FOR DATABASE OPERATIONS

    SET detail TO GET_SESSION_VALUE('detail')

    SET userId TO GET_SESSION_VALUE('userId')

    SET regNumber TO detail[0]

    SET make TO detail[4]

    SET type TO detail[-1]

    TRY

        EXECUTE SQL QUERY:

        "INSERT INTO tblCar VALUES (?, ?, ?, ?)"

        WITH VALUES (regNumber, make, type, userId)

        COMMIT CHANGES
```


©2025 Apple Garage Website

```
        CLOSE DATABASE CONNECTION
    CATCH EXCEPTION
        OUTPUT("This car already exists. Please enter a different car.")
```


©2025 Apple Garage Website

2.8. Book services

Screen design

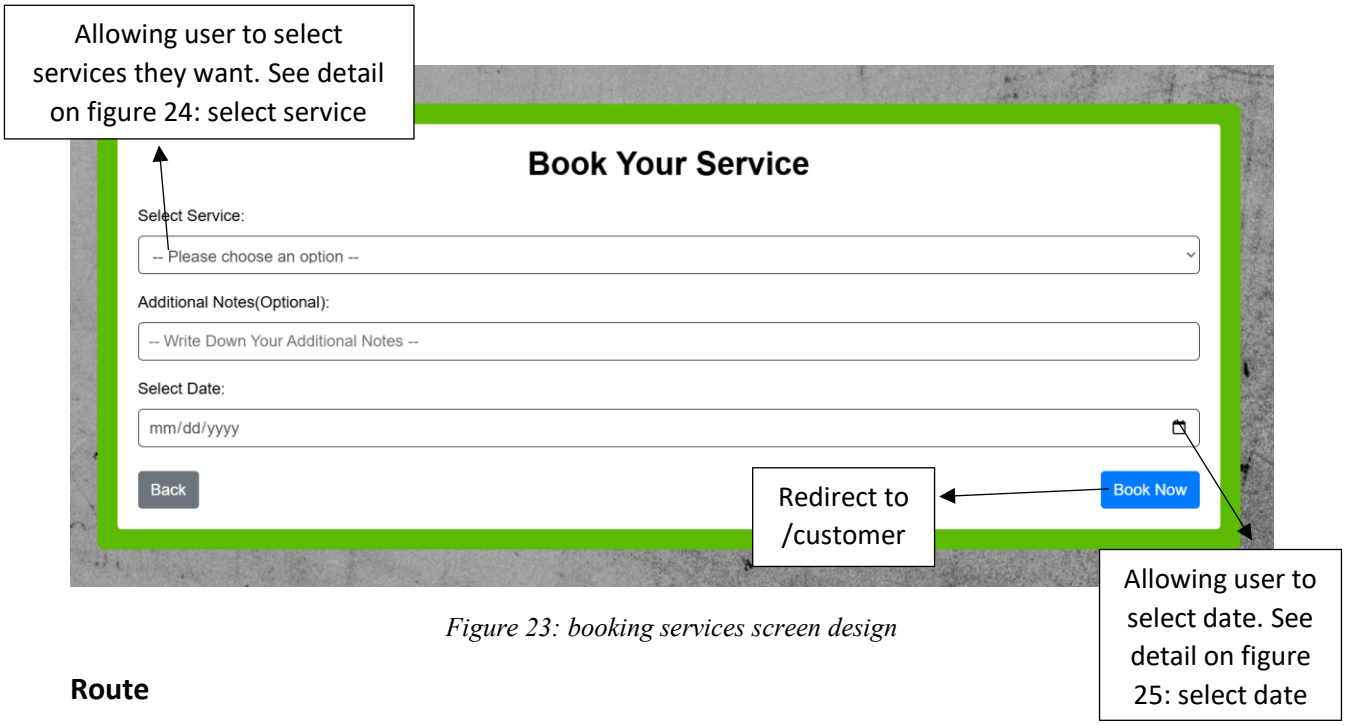


Figure 23: booking services screen design

Route

/booking

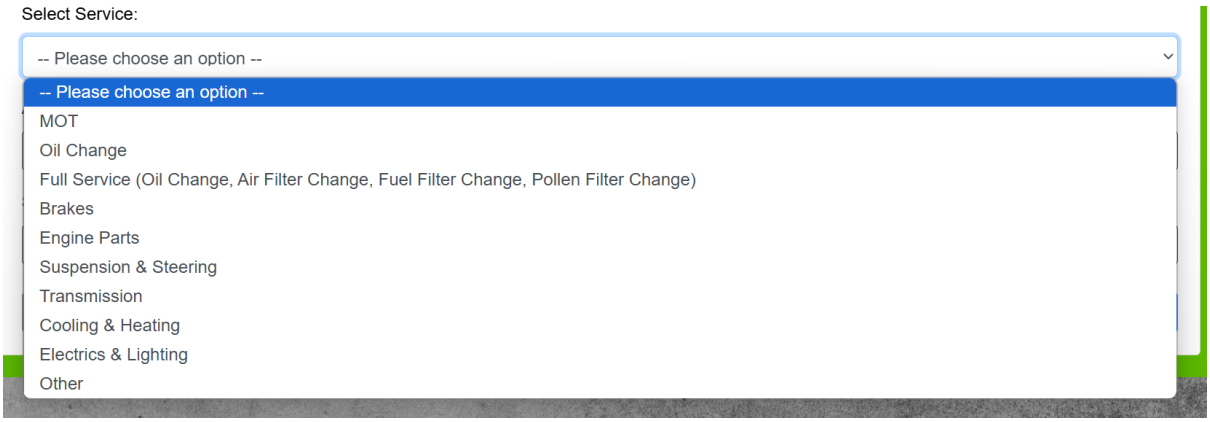


Figure 24: select service

©2025 Apple Garage Website

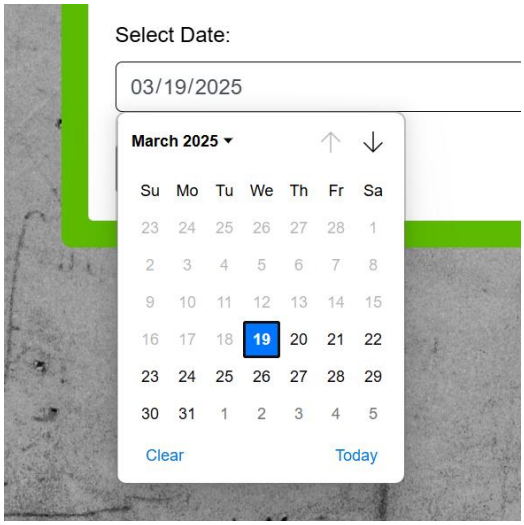


Figure 25: select date

Description

The book service page allows users to select service they want for their car and the date. User can also add additional note to describe their problem. Once all confirmed, user can press button 'book now' to book this service and add it to the scheduled list. The following SQL query uses to add booking into database:

```
INSERT INTO tblService

VALUES (?,?,?,?,?,?,?,?), (None, None, service, "Scheduled", date, None, reg, detail,))
```


©2025 Apple Garage Website

System flowchart

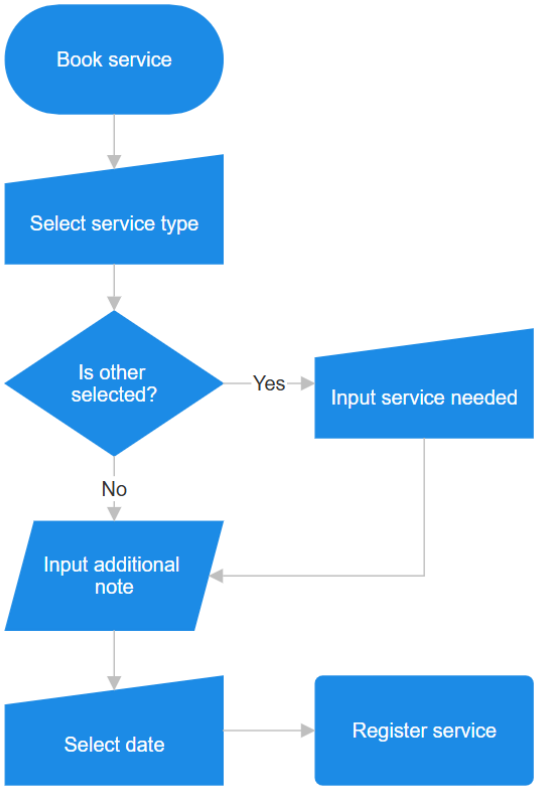


Figure 26: book service flowchart

©2025 Apple Garage Website

2.9. View car detail

Screen design

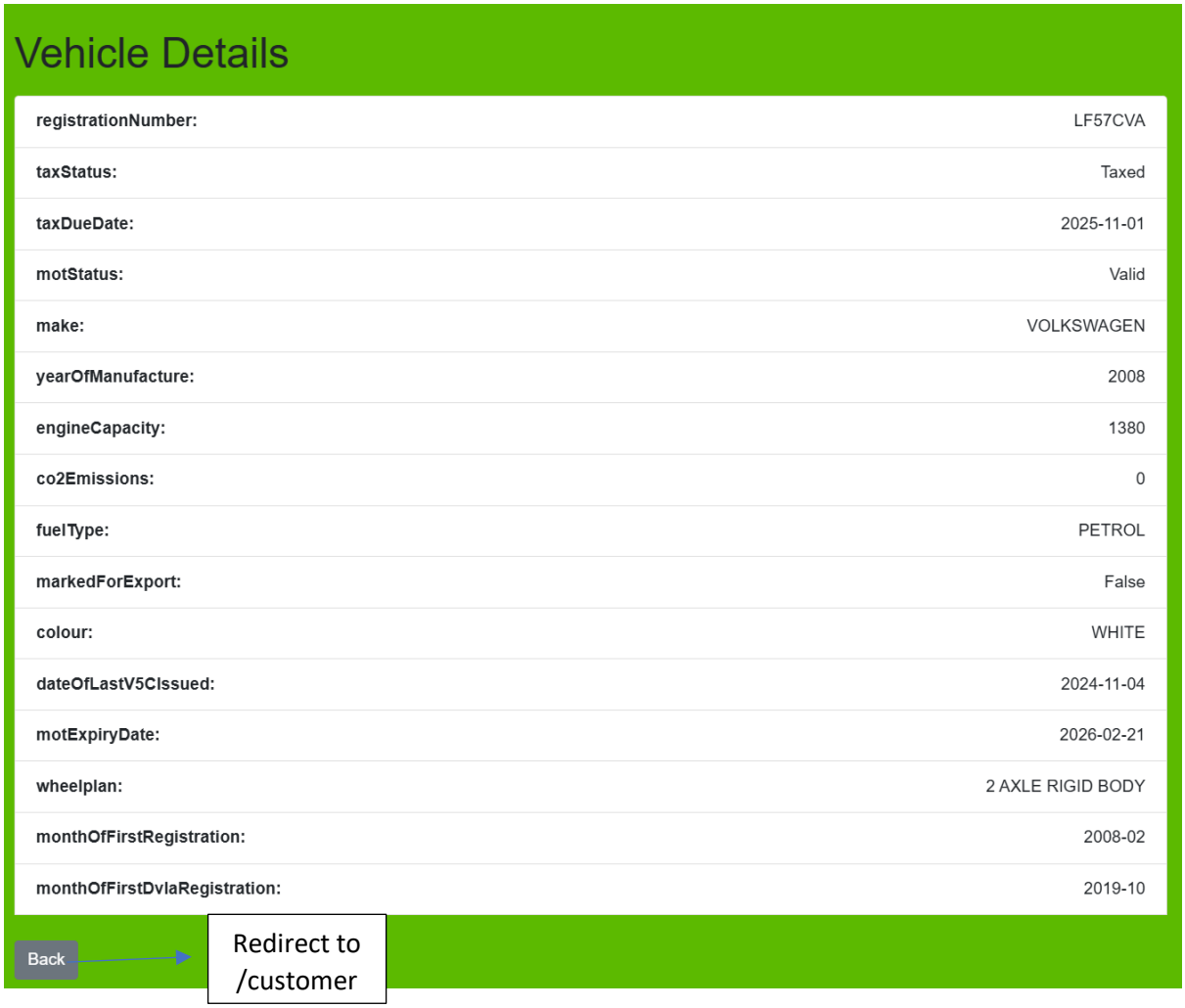


Figure 39: view car detail screen design

Route

/car_detail

Description

On this page customers can view their car details get from DVLA API at any time. The algorithm to get data form API and convert them into this format please see page 34 (Algorithm design – Retrieve vehicle details from API).

©2025 Apple Garage Website

2.10. Vehicle Maintenance record

Screen design

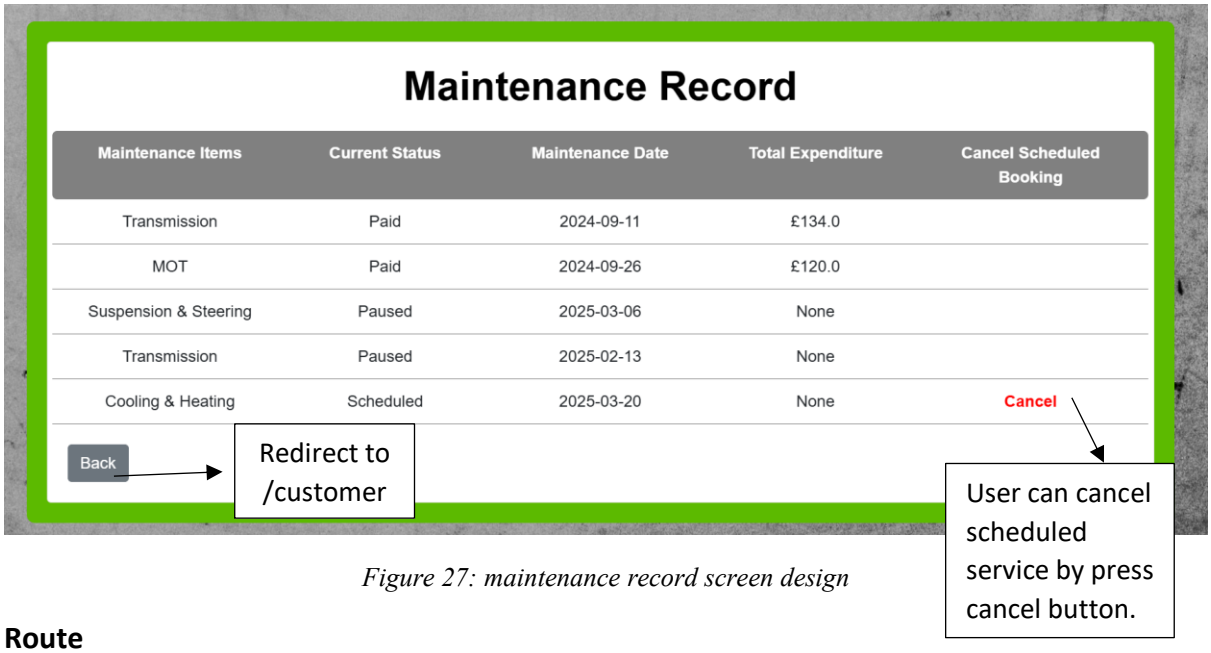


Figure 27: maintenance record screen design

Route

/record

Description

On this page users can see maintenance items, status, date and total expenditure for each service on single car. Status includes scheduled, working, paused, done and paid. Total expenditure will only display values if service was paid, otherwise it will display none. Cancel function will only available when service status was scheduled. The following SQL query uses to retrieve vehicle's maintenance record from database:

```
SELECT type, status, date, price, serviceId, regNumber
FROM tblService
WHERE regNumber = ?""", (reg,)
```


©2025 Apple Garage Website

2.11. Staff landing page

Screen design

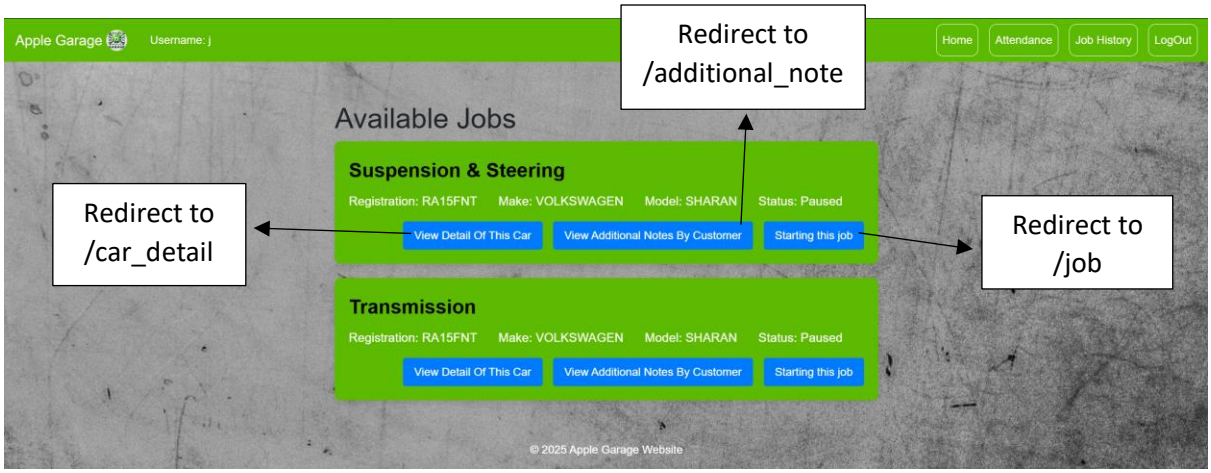


Figure 28: staff landing page screen design

Description

The staff landing page is similar to customer landing page. On the page will display all the available jobs instead of car. Staff can start any scheduled jobs or jobs paused by themselves. Before starting any job staff also able to view details of the car and any additional notes by customer.

Route

/staff

Staff system flowchart

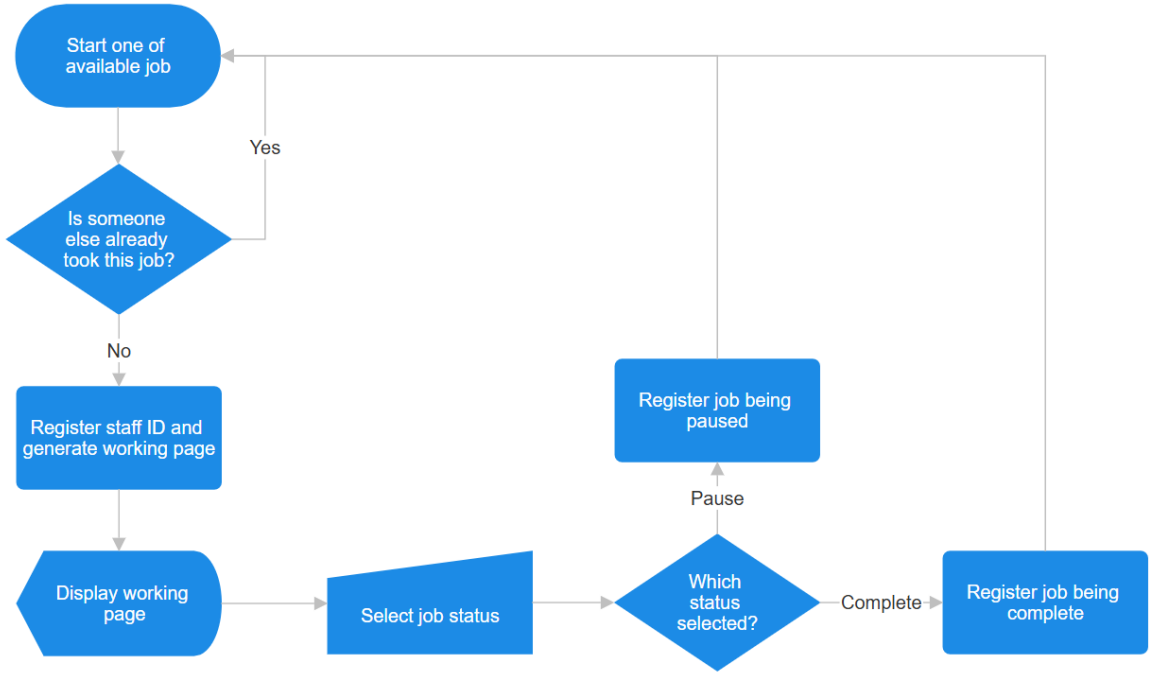


Figure 29: staff system flowchart

©2025 Apple Garage Website

2.12. Attendance

Screen design

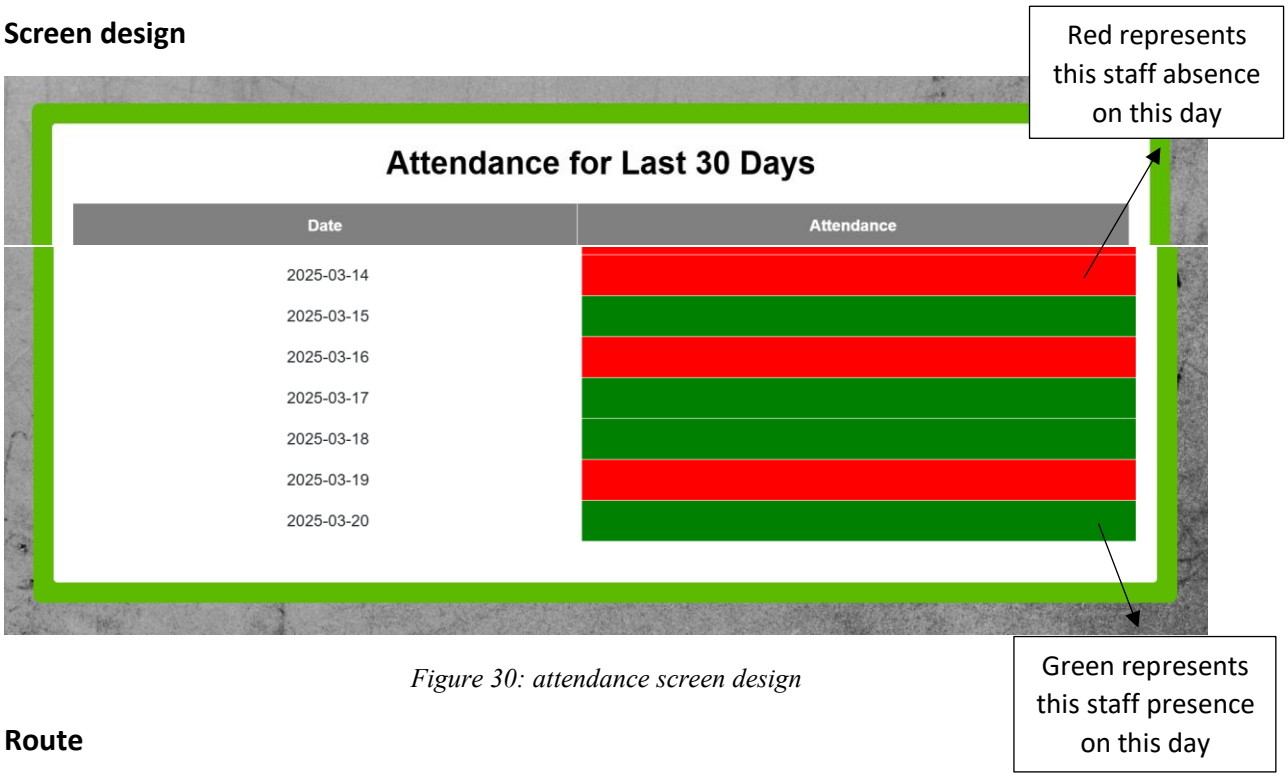


Figure 30: attendance screen design

Route

/attendance

Check in system flowcharts

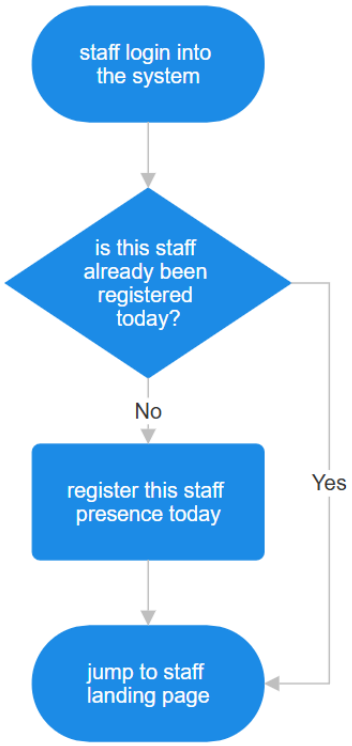


Figure 31: check in system flowcharts

©2025 Apple Garage Website

Description

Check in system was designed to work together with attendance page. It detects whether staff presence or absence on each day. The data check in system collected will be displayed on attendance page.

The following pseudo code is to get today's date and check whether user already has record in the database. If not will insert record into database.

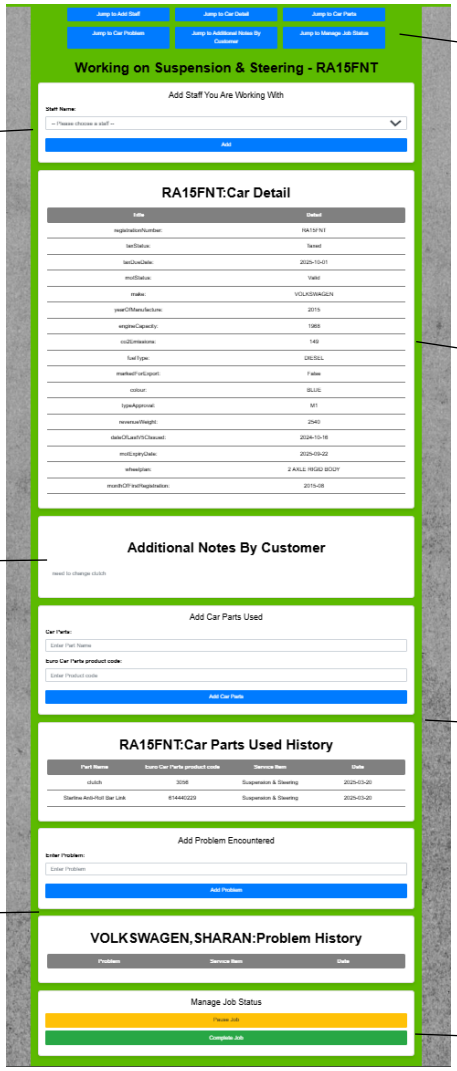
Algorithm design – check in system

```
    current_date = get today's date in format "YYYY-MM-DD"
    user_id = get value from session with key 'userId'
    connection = connect to database "carService.db"
    cursor = create a cursor from connection
    execute SQL query:
        "SELECT * FROM tblCheckIn WHERE userId = user_id AND date = 
current_date"
    result = fetch the first record from the query
    IF result exists THEN
        close the database connection
        redirect user to "/staff"
    ELSE
        "INSERT INTO tblCheckIn VALUES (NULL, user_id, current_date)"
        commit the changes to the database
        close the database connection
    END IF
```


©2025 Apple Garage Website

2.13. Working page

Screen design



Section allows staff add other staffs they are working with for this job. See detail of this section on page 45

Jump buttons allow user quick jump between sections

Section allows to view car detail. See detailed design of this section on page 39

Section allows to view additional notes by customer. See detail of this section on page 46

Section allows to add car part used and view car part used history on this car. See detail of this section on page 47

Section allows to write down any problem encountered and view any problem other staff encountered on same car model. See detail of this section on page 48

Buttons allow staff to pause or complete current job. See detail of this section on page 49

Figure 32: working page screen design

Description

To simplify operation while staff are doing their job. The working page was designed to be contain all functions. These functions are split into sections from top to bottom of the page. User only need to scroll up and down to access all the functions. They can also quick jump between each sections by click jump buttons at the top. The detail of each section will be explained in next few pages.

Route

/job

©2025 Apple Garage Website

2.14. Add staff working with

Screen design

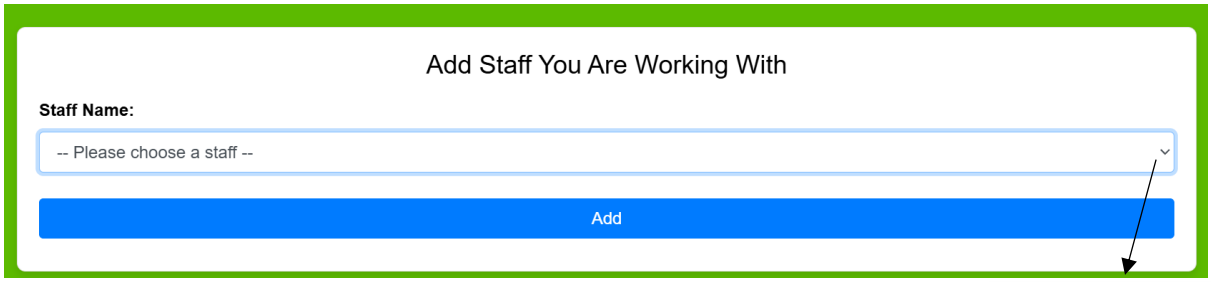


Figure 33: add staff screen design

Description

This function allows a staff to add other staff they are working with, in case multiple mechanics are working on the same car. Same staff name can only been added once.

Algorithm design – add staff working with

```
    OPEN connection to "carService.db"
    CREATE a cursor from the connection
    TRY:
        EXECUTE SQL query "INSERT INTO tblServiceStaff VALUES (serviceId,
staff)"
        COMMIT the transaction
        CLOSE the database connection
    CATCH any error:
        ROLLBACK the transaction
        CLOSE the database connection
        DISPLAY error message "This staff was already added."
        REDIRECT to the '/job' page
```


©2025 Apple Garage Website

2.15. View additional notes

Screen design

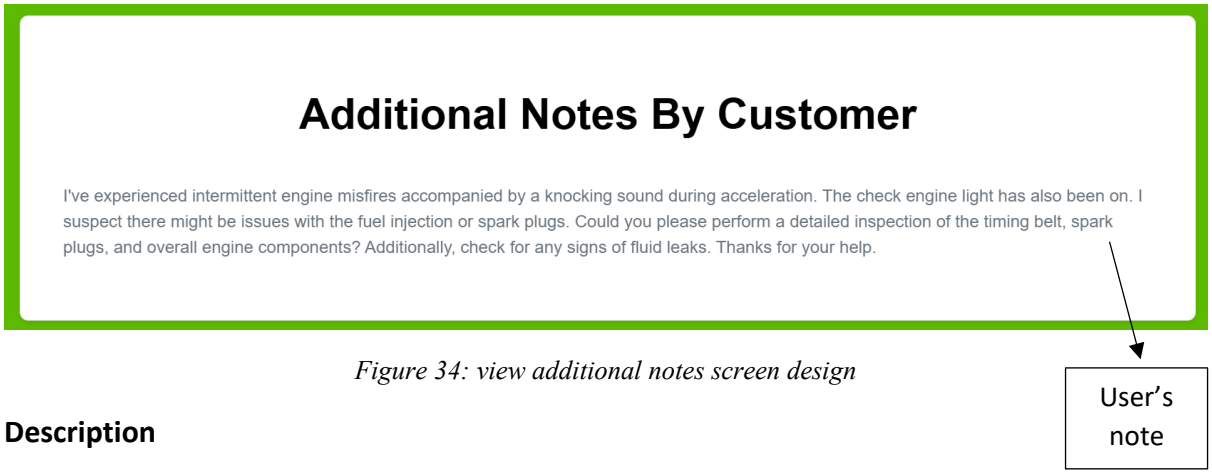


Figure 34: view additional notes screen design

Description

On this page will display customer's additional notes entered when booking the service. These notes will be retrieved from database by execute following SQL query:

```
SELECT detail(notes)
FROM tblService
WHERE serviceId = service_id
```


©2025 Apple Garage Website

2.16. Add car part

Screen design

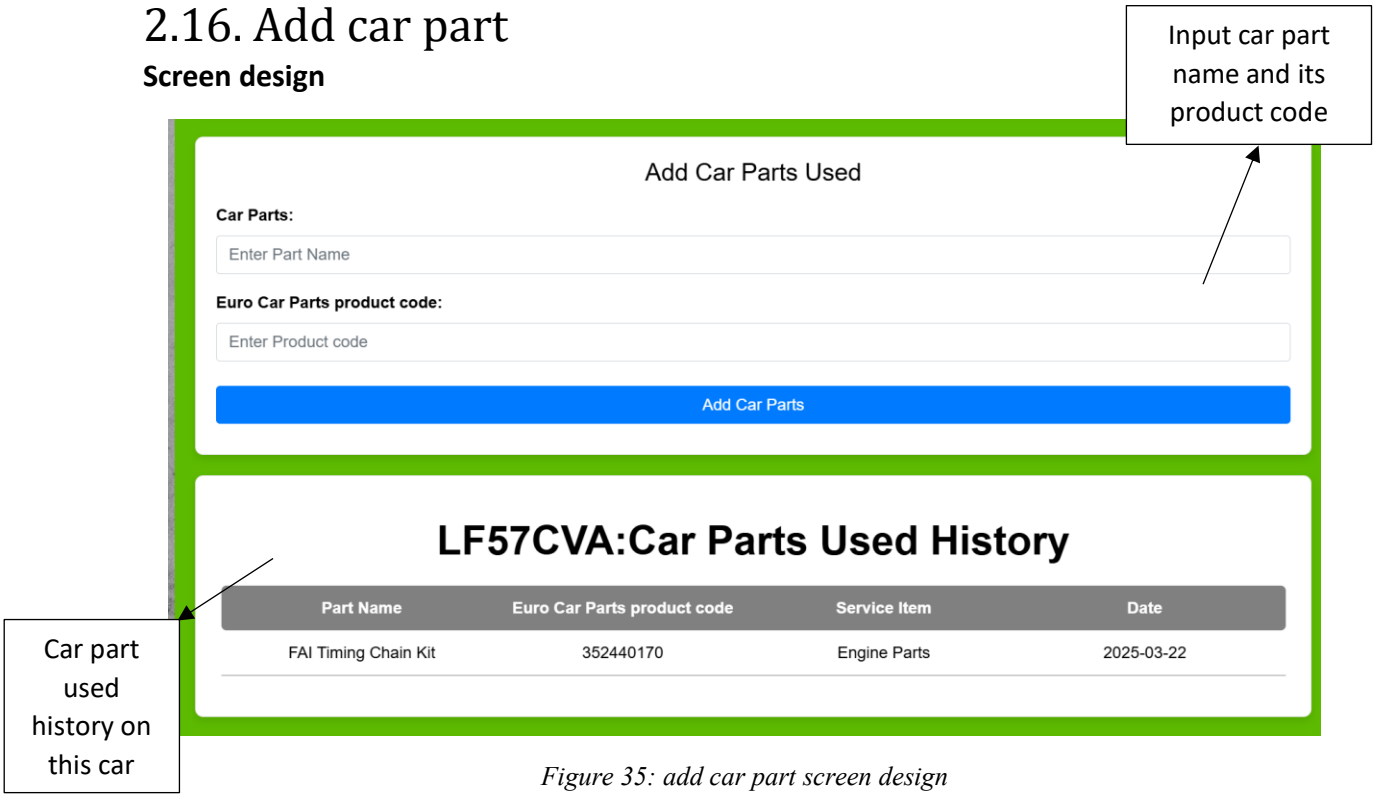


Figure 35: add car part screen design

Description

This section contains two parts, first part is for user to input the car part they used. Based on the research, the input box for product code also been added. It will help to find exact which car part used. The second part is to display all car parts used on this car. So is easier for mechanic to know which brand of car parts was used on this car.

SQL query to add car part used:

```
INSERT INTO tblParts
VALUES(?,?,?,?), (None, parts, serviceId, partId)
```

SQL query to retrieve car parts used history:

```
SELECT parts, type, date, partId
FROM tblService, tblParts
WHERE tblService.serviceId = tblParts.serviceId AND regNumber = ?, (reg,)
```


©2025 Apple Garage Website

2.17. Add problem encountered

Screen design

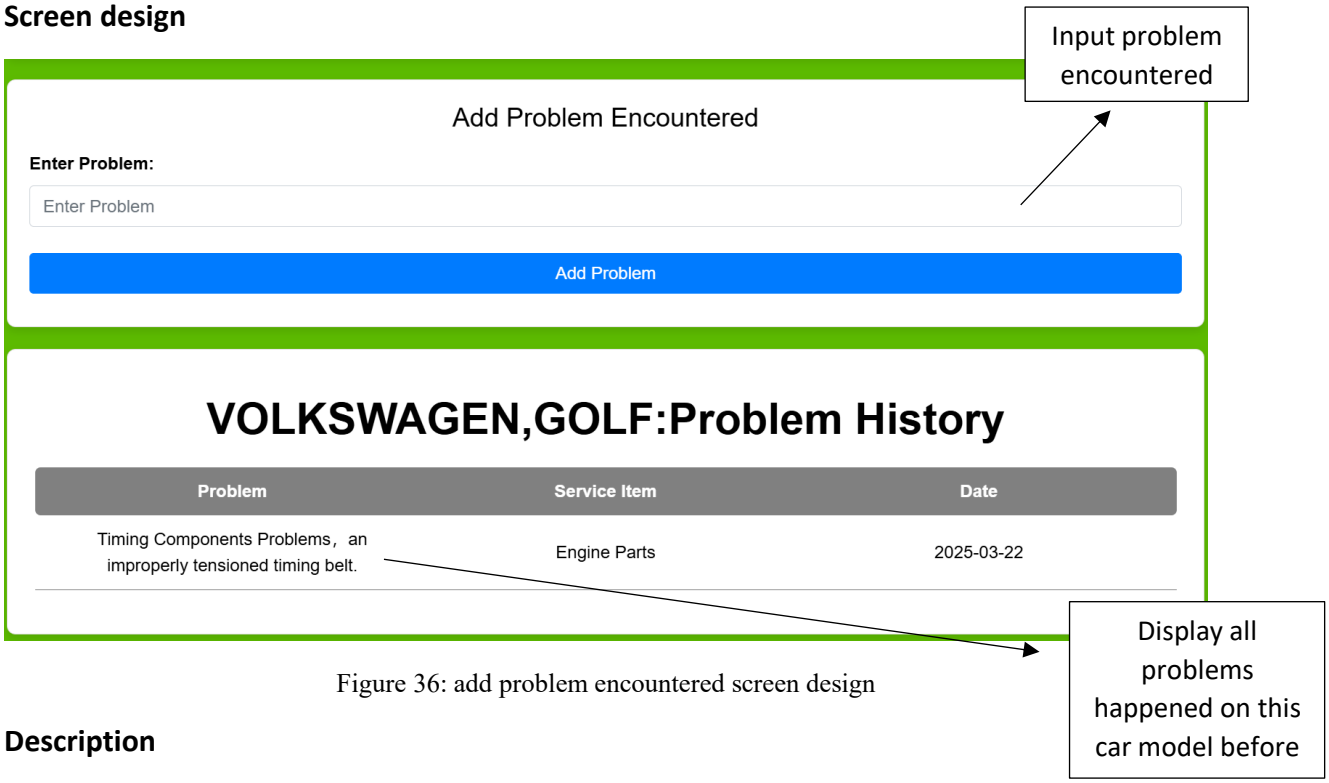


Figure 36: add problem encountered screen design

Description

This section is similar to add car part section. The first part is to add problem encountered. The second part is to display all problem happened on this model of car. The idea of this function is to allow mechanic to know potential problems on this model. Therefore, can save their time and improve service efficiency.

SQL query to add problem into database:

```
INSERT INTO tblProblem
VALUES(?,?,?), (None, problem, serviceId,)
```

SQL query to retrieved problem encountered history:

```
SELECT problem, type, date
FROM tblService, tblProblem, tblCar
WHERE tblService.serviceId = tblProblem.serviceId
AND regNumber = registrationNumber
AND carModel = ?
AND make = ?, (model, make,)
```


©2025 Apple Garage Website

2.18. Managing job status

Screen design

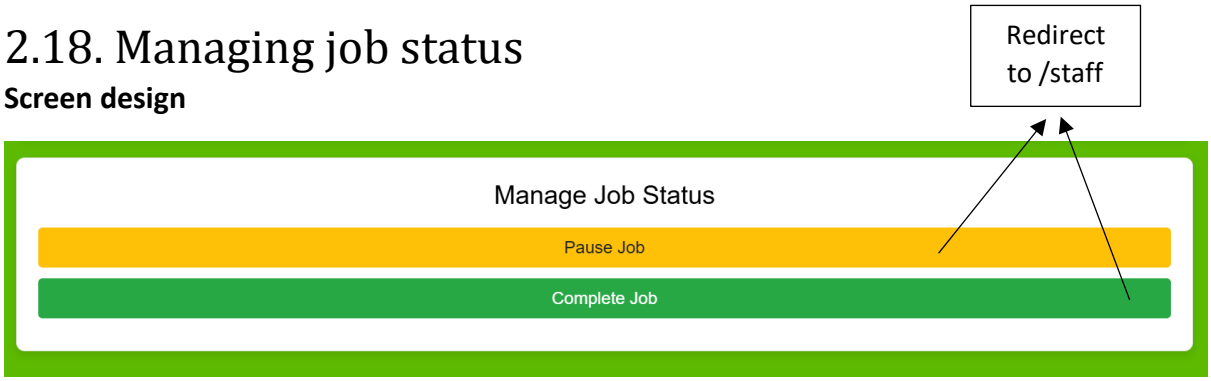


Figure 37: managing job status screen design

Description

Pause job button allows staff to pause the current job they are working on. They can go back to staff landing page and starting another job. The job been paused will stay on the available list. Staff can come back working on this job at any time. If choose complete job will remove from available list and sent to manager.

©2025 Apple Garage Website

2.19. Manager page

Screen design



Figure 38: manager page screen design

©2025 Apple Garage Website

Description

The manager page was designed to be a one big page which contains all services record sorted from newest to oldest. They are displayed in different colours which based on their status:

- Red – paused
- Green – done
- Yellow – working
- Blue – scheduled
- White – paid

This color-coding helps the manager quickly assess the status of every service. By clicking the View button on a record, the manager can see details such as the parts used, and any problems encountered. The Cancel button is only available for services that are scheduled, and the Enter Charge field becomes visible when a service is marked as completed.

Route

/manager

SQL query to retrieve all the service records with their details:

```
SELECT tblService.type, tblService.status, tblService.date, tblService.price,
tblService.serviceId, tblUser.userName,

GROUP_CONCAT(tblServiceStaff.staffName, ', ') as helpStaff, tblCar.make,
tblCar.carModel

FROM tblService

LEFT JOIN tblUser ON tblService.staffId = tblUser.userId

LEFT JOIN tblCar ON tblCar.registrationNumber = tblService.regNumber

LEFT JOIN tblServiceStaff ON tblService.serviceId = tblServiceStaff.serviceId

GROUP BY tblService.serviceId

ORDER BY tblService.date DESC
```


©2025 Apple Garage Website

2.20. Create staff account

This function allows manager to create new staff account. Create staff account page is similar to create customer account. It just changed user type from customer to staff. See detailed design on page 29 sign up.

Route

/create_staff_account

©2025 Apple Garage Website

2.21. Database design

Entity-relationship diagram

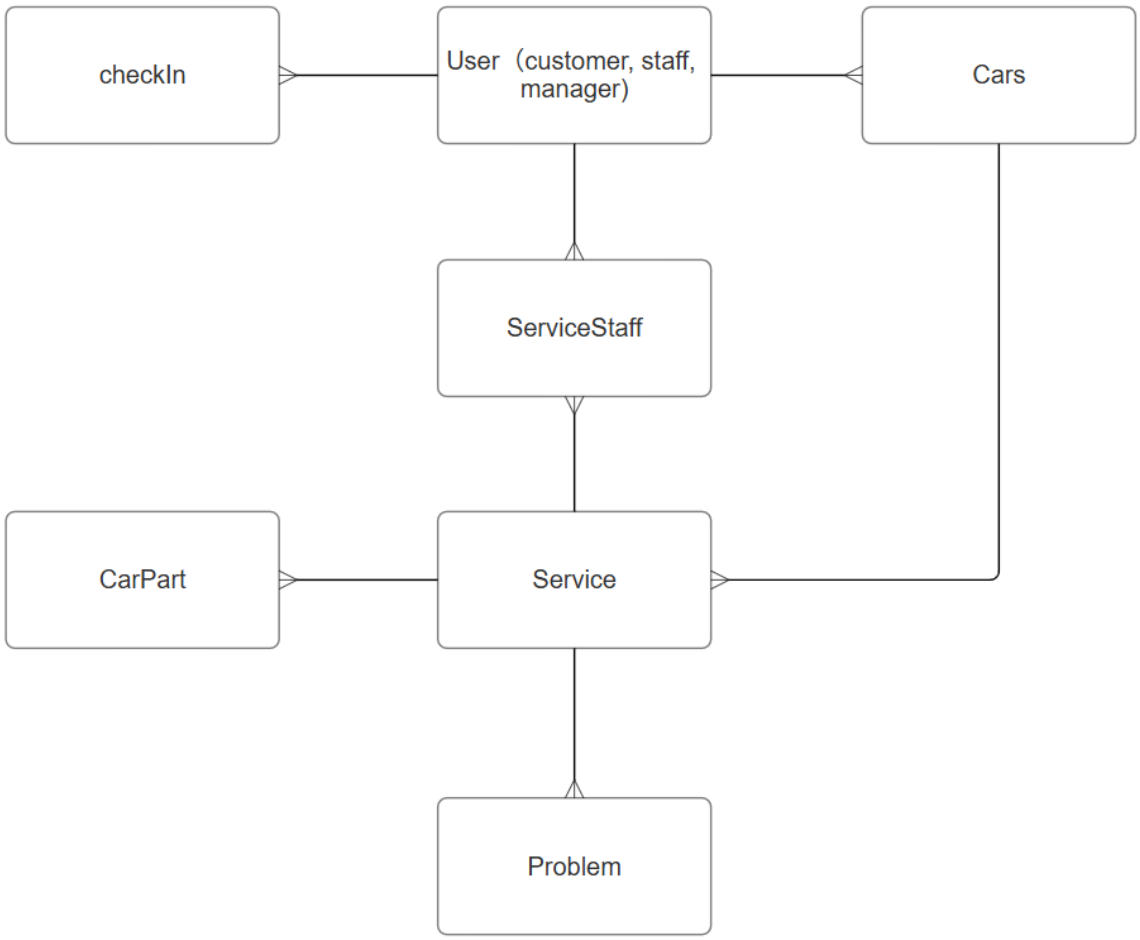


Figure 7: Entity-relationship diagram

- **Table User**

Field	Key	Data Type	Validation	Notes
userId	Primary	Integer		
userName		String	Unique	
userPassword		String		Will encrypt before store into database
userType		String		User type includes customer, staff and manager

©2025 Apple Garage Website

- **Table Car**

Field	Key	Data Type	Validation	Notes
registrationNumber	Primary	String		
make		String		All letters will be uppercase
carModel		String		All letters will be uppercase
userId	Foreign	Integer		Car's owner. Foreign key link to table user.

- **Table Service**

Field	Key	Data Type	Validation	Notes
serviceId	Primary	Integer		
staffId	Foreign	Integer		Foreign key link to table user.
type		String		
status		String		
date		String		YYYY-MM-DD
price		Real	>0	
regNumber	Foreign	String		Foreign key link to table car.
detail		String		Customer's additional notes

- **Table checkIn**

Field	Key	Data Type	Validation	Notes
id	Primary	Integer		
userId	Foreign	Integer		Foreign key link to table user.
date		String		YYYY-MM-DD

- **Table parts**

Field	Key	Data Type	Validation	Notes
id	Primary	Integer		
serviceId	Foreign	Integer		Foreign key link to table service.
partId		integer		Product code from https://www.eurocarparts.com/
parts		String		Car part used

©2025 Apple Garage Website

- Table problem

Field	Key	Data Type	Validation	Notes
id	Primary	Integer		
serviceId	Foreign	Integer		Foreign key link to table service.
problem		String		

- Table serviceStaff

Field	Key	Data Type	Validation	Notes
id	Primary	Integer		
serviceId	Foreign key	Integer		Foreign key link to table service.
staffName		String		Assist staff

©2025 Apple Garage Website

2.22. File structure

- Static
  - Images
    - Background.jpg
    - Logo.jpg
- Templates
  - Layout.html
  - Landing.html
  - Login.html
  - SignUp.html
  - ResetPassword.html
  - Customer.html
  - AddCar.html
  - Error.html
  - Detail.html
  - Car_detail.html
  - Booking.html
  - Record.html
  - Staff.html
  - Additional_note.html
  - Job.html
  - Attendance.html
  - jobHistory.html
  - Manager.html
  - serviceDetail.html
  - CreateStaffAccount.html
- carService.db
- main code.py

©2025 Apple Garage Website

3. Technical Solution

3.1. Python code

```
#set up: import libraries and modules
import requests
import os
import sqlite3
from flask import Flask, render_template, request, redirect, session, flash
import socket
from datetime import datetime, timedelta
import bcrypt

app = Flask(__name__)

# Set the secret key to secure sessions.
# The secret key is determined by the 'FLASK_SECRET_KEY' environment variable.
app.secret_key = os.environ.get('FLASK_SECRET_KEY', 'fallback_secret_key')


# Before each request, set username globally in template
@app.before_request
def display_username():
    username = session.get('username')
    if username:
        app.jinja_env.globals['username'] = username
    else:
        app.jinja_env.globals['username'] = None


# Landing page route
@app.route('/')
def landing():
    return render_template('landing.html')


# Login page route
@app.route('/login')
def login():
    return render_template('login.html')


# Logout route clear the session and redirects to the landing page
@app.route('/logout')
def logout():
    session.clear()
    return redirect('/')
```


©2025 Apple Garage Website

```
# Authentication route to process login authentication
@app.route('/authenticate', methods=["get", "post"])
def authenticate():
    # Connect to the database
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()

    # Get username and password from the form
    username = str(request.form['username'])
    password = str(request.form['password']).encode('utf-8')

    # Retrieve user details from the database based on username
    cursor.execute("""SELECT userID, userPassword, userType
                      FROM tblUser
                      WHERE tblUser.userName = ?""", (username,))
    authenticate_data = cursor.fetchone()

    # Check if user exists and provided password matches the password
    if authenticate_data and bcrypt.checkpw(password, authenticate_data[1]):
        session['userId'] = authenticate_data[0]
        session['username'] = username
        user_type = authenticate_data[2]
        conn.close()
        # Redirect based on user type
        if user_type == "staff":
            return redirect('/checkIn')
        elif user_type == "customer":
            return redirect('/customer')
        else:
            return redirect('/manager')
    else:
        conn.close()
        # Stay in login page with error message if authentication fails
        return render_template('login.html', error="Username or password is
wrong. Please try again!")


# Sign up page route
@app.route('/signUp')
def signUp():
    return render_template('signUp.html')


# Create account route: processes sign-up function
@app.route('/createAccount', methods=["get", "post"])
def createAccount():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
```


©2025 Apple Garage Website

```
    username = str(request.form['username'])
    password = str(request.form['password']).encode('utf-8')
    # Hash the password using bcrypt
    hashed_password = bcrypt.hashpw(password, bcrypt.gensalt())

    # Insert a new customer record; show error if username already exists
    try:
        cursor.execute("INSERT INTO tblUser VALUES(?,?,?,?)", (None, username,
hashed_password, "customer"))
    except:
        conn.rollback()
        return render_template("signUp.html", error="Username already exists.
Please choose a different username.")
    conn.commit()
    conn.close()
    return redirect('/login')


# Reset password page route
@app.route('/resetPassword')
def resetPassword():
    return render_template('resetPassword.html')


# Change password route
@app.route('/changePassword', methods=["get", "post"])
def changePassword():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()

    # Get current username, current password and new password from the form
    username = str(request.form['username'])
    password = str(request.form['password']).encode('utf-8')
    newPassword = str(request.form['newPassword']).encode('utf-8')
    hashed_password = bcrypt.hashpw(newPassword, bcrypt.gensalt())

    # Verify current password
    cursor.execute("""SELECT userPassword
                      FROM tblUser
                      WHERE tblUser.userName = ?""", (username,))
    authenticate_data = cursor.fetchone()
    if authenticate_data and bcrypt.checkpw(password, authenticate_data[0]):
        # Update password in the database if current password is correct
        cursor.execute("UPDATE tblUser SET userPassword = ? WHERE userName
= ?", (hashed_password,username, ))
        conn.commit()
        conn.close()
        return redirect('/login')
    else:
```


©2025 Apple Garage Website

```
        conn.close()
        return render_template('resetPassword.html', error="Username or
password is wrong. Please try again!")


# Add car page route
@app.route('/addCar')
def addCar():
    return render_template('addCar.html')


# Route to process car registration number and fetch vehicle data from DVLA
API
@app.route('/regNumber', methods=['GET', 'POST'])
def regNumber():
  url = "https://driver-vehicle-licensing.api.gov.uk/vehicle-
enquiry/v1/vehicles"
  reg = str(request.form['regNumber'])
  payload = {"registrationNumber": reg}
  headers = {
    'x-api-key': 'ciMvGeHcY85HvJJsO2XPy1BN8hX8ZAaK6wb03hGl',
    'Content-Type': 'application/xml'
  }
  response = requests.post(url, headers=headers, json=payload)

  # Extract keys (titles) from API response and store in session
  title = []
  for i in response.json():
    title.append(i)
  session['title'] = title

  # Extract values (details) from API response and store in session
  detail = []
  for i in response.json().values():
    detail.append(i)
  session['detail'] = detail

  return redirect('/detail')


# Display vehicle details route and allows to input car model
@app.route('/detail', methods=['GET', 'POST'])
def detail():
  detail = session.get('detail')
  title = session.get('title')
  num = len(detail)

  # If the API returns an error, redirect to error page
  if title[0] == "errors":
```


©2025 Apple Garage Website

```
    return render_template('error.html')

  # If a POST request with car model is made, update the details
  if request.method == 'POST':
        model = request.form.get('model')
        if model:
            model = model.upper()
            detail.append(model)
            title.append("model")
            session['detail'] = detail
            session['title'] = title
            return redirect('/detail')
  return render_template('detail.html', detail=detail, title=title, num=num)


# Route to store the vehicle details into the database
@app.route('/store_detail', methods=['GET', 'POST'])
def store_detail():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()

    detail = session.get('detail')
    userId = session.get('userId')
    regNumber = detail[0]
    make = detail[4]
    type = detail[-1]
    # Insert the new car into tblCar, preventing duplicates
    try:
      cursor.execute('''INSERT INTO tblCar VALUES (?,?,?,?)''', (regNumber,
make, type, userId))
      conn.commit()
      conn.close()
    except:
       conn.rollback()
       return render_template("addCar.html", error="This car already exists.
Please enter a different car.")
    return redirect('/customer')


# Customer landing page route
@app.route('/customer', methods=["get","post"])
def customer():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    userId = session.get('userId')

    car = cursor.execute("""SELECT * FROM tblCar WHERE userId = ?""",
(userId,))
    listOfCar = []
```


©2025 Apple Garage Website

```
    for each in car:
        listOfCar.append(each)

    conn.close()
    return render_template('customer.html', car=listOfCar)


# Route to display vehicle detail for a specific car using its registration
number
@app.route('/car_detail/<reg>', methods=["get","post"])
def car_detail(reg):
  url = "https://driver-vehicle-licensing.api.gov.uk/vehicle-
enquiry/v1/vehicles"
  payload = {"registrationNumber": reg}
  headers = {
    'x-api-key': 'ciMvGeHcY85HvJJsO2XPy1BN8hX8ZAaK6wb03hGl',
    'Content-Type': 'application/xml'
  }
  response = requests.post(url, headers=headers, json=payload)
  title = []
  for i in response.json():
    title.append(i)
  detail = []
  for i in response.json().values():
    detail.append(i)
  num = len(title)
  return render_template('car_detail.html', detail=detail, title=title,
num=num)


# Route to delete car
@app.route('/delete_car/<reg>', methods=["get","post"])
def delete_car(reg):
    userId = session.get('userId')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""DELETE FROM tblCar WHERE userId = ? AND
registrationNumber = ?""", (userId, reg,))
    conn.commit()
    conn.close()
    return redirect('/customer')


# Booking page route
@app.route('/booking/<reg>', methods=["get","post"])
def booking(reg):
    today = datetime.today().strftime('%Y-%m-%d')
    return render_template('booking.html', reg=reg, today=today)
```


©2025 Apple Garage Website

```
# Route to add a booking record into database
@app.route('/add_booking/<reg>', methods=["get","post"])
def add_booking(reg):
    service = request.form.get('service')
    detail = request.form.get('detail')
    other_service = request.form.get('otherService')
    date = request.form.get('date')

    # Use user's input if "Other" is selected
    if service == 'Other':
        service = other_service

    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute('''INSERT INTO tblService VALUES (?,?,?,?,?,?,?,?)''',
(None, None, service, "Scheduled", date, None, reg, detail,))
    conn.commit()
    conn.close()

    return redirect(f'/record/{reg}')


# Route to display service records for a particular car
@app.route('/record/<reg>', methods=["get","post"])
def record(reg):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    record = cursor.execute("""SELECT type, status, date, price, serviceId,
regNumber FROM tblService WHERE regNumber = ?""", (reg,))
    listOfRecord = []
    for each in record:
        listOfRecord.append(each)
    return render_template('record.html', record=listOfRecord)


# Route to delete a service record if status is scheduled
@app.route('/delete_record/<reg>/<id>', methods=["get","post"])
def delete_record(reg, id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""DELETE FROM tblService WHERE serviceId = ?""", (id,))
    conn.commit()
    conn.close()
    return redirect(f'/record/{reg}')


# Staff check-in route: prevents multiple check-ins on the same day
@app.route('/checkIn', methods=["get","post"])
def checkIn():
```


©2025 Apple Garage Website

```
    today = datetime.today().strftime('%Y-%m-%d')
    userId = session.get('userId')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""SELECT * FROM tblCheckIn WHERE userId = ? AND date
= ?""", (userId, today,))
    result = cursor.fetchone()
    # If already checked in today, redirect to staff landing page
    if result:
        conn.close()
        return redirect('/staff')
    else:
        cursor.execute("INSERT INTO tblCheckIn VALUES(?,?,?)", (None, userId,
today,))
        conn.commit()
        conn.close()
        return redirect('/staff')


# Staff landing page route
@app.route('/staff', methods=["get","post"])
def staff():
    today = datetime.today().strftime('%Y-%m-%d')
    staffId = session.get('userId')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    # Fetch scheduled jobs on or before today
    job = cursor.execute("""SELECT regNumber, type, make, carModel, serviceId,
status
                            FROM tblService, tblCar
                            WHERE date <= ? AND status = "Scheduled" AND
registrationNumber = regNumber""", (today,))
    listOfJob = []
    for each in job:
        listOfJob.append(each)
    # Fetch jobs that have been paused by current staff
    waiting = cursor.execute("""SELECT regNumber, type, make, carModel,
serviceId, status
                                FROM tblService, tblCar
                                WHERE date <= ? AND status = "Paused" AND
registrationNumber = regNumber AND staffId = ?""", (today, staffId,))
    for each in waiting:
        listOfJob.append(each)
    conn.close()
    return render_template('staff.html', job=listOfJob, id=staffId)


# Route to display additional notes by customer
@app.route('/additional_notes/<id>', methods=["get","post"])
```


©2025 Apple Garage Website

```
def additional_notes(id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""SELECT detail FROM tblService WHERE serviceId = ?""",
(id,))
    result = cursor.fetchone()
    detail = result[0]
    conn.close()
    return render_template('additional_notes.html', detail=detail)


# Route to assign a staff member to a service job and mark its status to
'Working'
@app.route('/store_staffId/<reg>/<id>/<make>', methods=["get","post"])
def store_staffId(reg, id, make):
    session['reg'] = reg
    session['serviceId'] = id
    session['make'] = make
    staffId = session.get('userId')
    today = datetime.today().strftime('%Y-%m-%d')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("SELECT staffId FROM tblService WHERE serviceId = ?",
(id,))
    result = cursor.fetchone()
    # Prevent multiple staff members taking the same job
    if result[0] is not None and result[0] != staffId:
        conn.close()
        return render_template("staff.html", error="Someone already take this
job. Please choose a different job or work together with this person.")
    cursor.execute("""UPDATE tblService SET staffId = ?, status = 'Working',
date = ? WHERE serviceId = ?""", (staffId, today, id,))
    conn.commit()
    conn.close()
    return redirect('/job')


# Working page route
@app.route('/job', methods=["get","post"])
def job():
    reg = session.get('reg')
    make = session.get('make')
    serviceId = session.get('serviceId')
    staffId = session.get('userId')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()

    # Retrieve the type of service for the current job
```


©2025 Apple Garage Website

```
    cursor.execute("""SELECT type FROM tblService WHERE serviceId = ?""",
(serviceId,))
    result = cursor.fetchone()
    type = result[0]

    # Retrieve parts used for this service
    parts = cursor.execute("""SELECT parts, type, date, partId
                              FROM tblService, tblParts
                              WHERE tblService.serviceId = tblParts.serviceId
AND regNumber = ?""", (reg,))
    listOfParts = []
    for each in parts:
        listOfParts.append(each)

    # Get car model
    cursor.execute("""SELECT carModel FROM tblCar WHERE registrationNumber
= ?""", (reg,))
    result = cursor.fetchone()
    model = result[0]

    # Retrieve problems encountered for same model's car
    problem = cursor.execute("""SELECT problem, type, date
                                FROM tblService, tblProblem, tblCar
                                WHERE tblService.serviceId =
tblProblem.serviceId
                                AND regNumber = registrationNumber
                                AND carModel = ?
                                AND make = ?""", (model, make,))
    listOfProblem = []
    for each in problem:
        listOfProblem.append(each)

    # Get additional notes from customer
    cursor.execute("""SELECT detail FROM tblService WHERE serviceId = ?""",
(serviceId,))
    result = cursor.fetchone()
    note = result[0]

    # Retrieve list of other staff members
    staff = cursor.execute("""SELECT userName FROM tblUser WHERE userType =
'staff' AND userId != ?""", (staffId,))
    listOfStaff = []
    for each in staff:
        listOfStaff.append(each[0])

    # Fetch vehicle information from the DVLA API
    url = "https://driver-vehicle-licensing.api.gov.uk/vehicle-
enquiry/v1/vehicles"
```


©2025 Apple Garage Website

```
    payload = {"registrationNumber": reg}
    headers = {
      'x-api-key': 'ciMvGeHcY85HvJJsO2XPy1BN8hX8ZAaK6wb03hGl',
      'Content-Type': 'application/xml'
    }
    response = requests.post(url, headers=headers, json=payload)
    title = []
    for i in response.json():
      title.append(i)
    detail = []
    for i in response.json().values():
      detail.append(i)
    num = len(title)
    conn.close()
    return render_template('job.html', reg=reg, serviceId=serviceId,
type=type, parts=listOfParts, problem=listOfProblem, model=model, make=make,
detail=detail, title=title, num=num, note=note, staff=listOfStaff)


# Route to pause current service job
@app.route('/pause/<id>', methods=["get","post"])
def pause(id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""UPDATE tblService SET status = 'Paused' WHERE serviceId
= ?""", (id,))
    conn.commit()
    conn.close()
    return redirect('/staff')


# Route to add parts used during the job
@app.route('/addParts', methods=["get", "post"])
def addParts():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    parts = request.form['parts']
    partId = request.form['partId']
    serviceId = session.get('serviceId')
    cursor.execute("INSERT INTO tblParts VALUES(?,?,?,?)", (None, parts,
serviceId, partId))
    conn.commit()
    conn.close()
    return redirect('/job')


# Route to add a problem encountered during the job
@app.route('/addProblem', methods=["get", "post"])
def addProblem():
```


©2025 Apple Garage Website

```
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    problem = request.form['problem']
    serviceId = session.get('serviceId')
    cursor.execute("INSERT INTO tblProblem VALUES(?,?,?)", (None, problem,
serviceId,))
    conn.commit()
    conn.close()
    return redirect('/job')


# Route to add another staff member working with
@app.route('/addStaff', methods=["GET", "POST"])
def addStaff():
    if request.method == "POST":
        staff = request.form.get('staff')
        serviceId = session.get('serviceId')
        conn = sqlite3.connect("carService.db")
        cursor = conn.cursor()
        # Prevent adding the same staff member twice
        try:
            cursor.execute("INSERT INTO tblServiceStaff VALUES (?, ?, ?)",
(None, serviceId, staff))
            conn.commit()
            conn.close()
        except:
            conn.rollback()
            conn.close()
            flash("This staff was already added.", "error")
            return redirect('/job')
    return redirect('/job')


# Route to mark current job complete
@app.route('/complete/<id>', methods=["get","post"])
def complete(id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""UPDATE tblService SET status = 'Done' WHERE serviceId
= ?""", (id,))
    conn.commit()
    conn.close()
    return redirect('/staff')


# Route to display attendance records for the last 30 days
@app.route('/attendance/<id>', methods=["get","post"])
def attendance(id):
    conn = sqlite3.connect("carService.db")
```


©2025 Apple Garage Website

```
    cursor = conn.cursor()
    cursor.execute("""SELECT date FROM tblCheckIn WHERE userId = ?""", (id,))
    records = cursor.fetchall()
    listOfDate = [record[0] for record in records]
    today = datetime.today()
    new_date = today - timedelta(days=29)
    date = new_date.strftime('%Y-%m-%d')
    thirty_days_list = []
    color = []
    # For each day, mark green if present, red if absent
    for i in range(30):
        thirty_days_list.append(date)
        if date in listOfDate:
            color.append("green")
        else:
            color.append("red")
        new_date = today - timedelta(days=29-(i+1))
        date = new_date.strftime('%Y-%m-%d')
    conn.close()
    date_color = zip(thirty_days_list, color)
    return render_template('attendance.html', date_color=date_color, id=id)


# Route to display the job history
@app.route('/jobHistory', methods=["get","post"])
def jobHistory():
    staffId = session.get('userId')
    id = session.get('userId')
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    record = cursor.execute("""SELECT type, status, date, make, carModel,
regNumber
                               FROM tblService, tblCar
                               WHERE regNumber = registrationNumber AND
staffId = ?""", (id,))
    listOfRecord = record.fetchall()
    return render_template('jobHistory.html', record=listOfRecord, id=staffId)


# Manager landing page route
@app.route('/manager', methods=["GET", "POST"])
def manager():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    record = cursor.execute("""SELECT tblService.type, tblService.status,
tblService.date, tblService.price, tblService.serviceId, tblUser.userName,
GROUP_CONCAT(tblServiceStaff.staffName, ', ') as helpStaff, tblCar.make,
tblCar.carModel
                               FROM tblService
```


©2025 Apple Garage Website

```
                                LEFT JOIN tblUser ON tblService.staffId =
tblUser.userId
                                LEFT JOIN tblCar ON tblCar.registrationNumber =
tblService.regNumber
                                LEFT JOIN tblServiceStaff ON
tblService.serviceId = tblServiceStaff.serviceId
                                GROUP BY tblService.serviceId
                                ORDER BY tblService.date DESC""")
    listOfRecord = record.fetchall()
    status = cursor.execute("""SELECT tblService.status
                                FROM tblService
                                ORDER BY tblService.date DESC""")
    listOfStatus = status.fetchall()
    # Assign a color to each record based on its status
    color = []
    for status in listOfStatus:
        if status[0] == "Scheduled":
            color.append("#add8e6")
        elif status[0] == "Working":
            color.append("orange")
        elif status[0] == "Paused":
            color.append("red")
        elif status[0] == "Done":
            color.append("green")
        elif status[0] == "Paid":
            color.append("white")
    conn.close()
    record_color = zip(listOfRecord, color)
    return render_template('manager.html', record_color=record_color)


# Route to delete a service record if status is scheduled
@app.route('/deleteRecord/<id>', methods=["get","post"])
def deleteRecord(id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    cursor.execute("""DELETE FROM tblService WHERE serviceId = ?""", (id,))
    conn.commit()
    conn.close()
    return redirect('/manager')


# Route to update the price and mark the service status as "Paid"
@app.route('/enterPrice/<id>', methods=["get","post"])
def enterPrice(id):
    price = str(request.form['price'])
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
```


©2025 Apple Garage Website

```
    cursor.execute("""UPDATE tblService SET price = ?, status = "Paid" WHERE
serviceId = ?""", (price, id,))
    conn.commit()
    conn.close()
    return redirect('/manager')


# Route to display service detail including car parts used and problem
encountered
@app.route('/serviceDetail/<id>', methods=["get","post"])
def carPartsService(id):
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    parts = cursor.execute("""SELECT parts FROM tblParts WHERE serviceId
= ?""", (id,))
    listOfParts = parts.fetchall()
    porblems = cursor.execute("""SELECT problem FROM tblProblem WHERE
serviceId = ?""", (id,))
    listOfProblems = porblems.fetchall()
    return render_template('serviceDetail.html', parts=listOfParts,
problems=listOfProblems)


# Create staff account page route
@app.route('/createStaffAccount')
def createStaffAccount():
    return render_template('createStaffAccount.html')


# Route to process create staff account function
@app.route('/create_Staff_Account', methods=["get", "post"])
def create_Staff_Account():
    conn = sqlite3.connect("carService.db")
    cursor = conn.cursor()
    username = str(request.form['username'])
    password = str(request.form['password']).encode('utf-8')
    hashed_password = bcrypt.hashpw(password, bcrypt.gensalt())
    # Insert new staff record; if duplicate username occurs, display error
    try:
       cursor.execute("INSERT INTO tblUser VALUES(?,?,?,?)", (None, username,
hashed_password, "staff"))
    except:
       conn.rollback()
       return render_template("createStaffAccount.html", error="Username
already exists. Please enter a different username.")
    conn.commit()
    conn.close()
    return redirect('/createStaffAccount')
```


©2025 Apple Garage Website

```
# Main entry point
if __name__ == "__main__":
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(('8.8.8.8', 1))
    local_ip_address = s.getsockname()[0]
    print("Local IP Address :", local_ip_address)
    app.run(host="0.0.0.0")
```


©2025 Apple Garage Website

3.2. HTML code

Layout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Apple Garage</title>
    <!-- Title of the page as shown in the browser tab -->
    <link
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css
" rel="stylesheet">
    <!-- Loads Bootstrap CSS for responsive design and prebuilt styles -->
    <style>
        /* Custom styles for the page */
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #3a4d4a;
        }
        footer {
            text-align: center;
            padding: 10px;
            color: white;
        }
        .navbar-custom {
            background-color: #5cba00;
            color: white;
        }
        .navbar-custom .navbar-brand,
        .navbar-custom .nav-link,
        .navbar-custom .nav-link:hover,
        .navbar-custom .navbar-toggler-icon {
            color: white;
        }
        body {
            background-image: url('/static/images/background.jpg');
            background-repeat: no-repeat;
            background-attachment: fixed;
            background-size: cover;
        }
        .content {
            padding-top: 70px;
        }
        .navbar-nav .nav-item {
            margin-left: 15px;
            border: 1px solid white;
            border-radius: 10px;
```


©2025 Apple Garage Website

```
        }
        .navbar-nav .nav-link {
            padding: 10px 15px;
        }
        .navbar-nav .nav-link:hover {
            background-color: #4c9a00;
            border-radius: 10px;
        }
    </style>
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-custom fixed-top">
        <a class="navbar-brand" href="">
            Apple Garage
            <img src="{{ url_for('static', filename='images/logo.png') }}"
alt="Car Service Logo" width="30" height="30" class="d-inline-block align-top"
style="border-radius: 50%;">
            <!-- The image is styled to appear as a circle (border-radius:
50%) -->
        </a>
        {% if username %}
            <span class="navbar-text text-white ml-3">Username:
{{ username }}</span>
        {% endif %}
        <!-- Collapsible part of the navbar -->
        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav ml-auto">
                {% block navbar %}
                <li class="nav-item">
                    <a id="navHome" class="nav-link" href="/">Home</a>
                </li>
                <li class="nav-item">
                    <a id="navLogin" class="nav-link" href="/login">LogIn</a>
                </li>
                <li class="nav-item">
                    <a id="navSignup" class="nav-link"
href="/signUp">SignUp</a>
                </li>
                <li class="nav-item">
                    <a id="navCustomer" class="nav-link" href="/customer">My
Car</a>
                </li>
                <li class="nav-item">
                    <a id="navDetail" class="nav-link"
href="/detail">Detail</a>
                </li>
                <li class="nav-item">
                    <a id="navError" class="nav-link" href="/error">Error</a>
```


©2025 Apple Garage Website

```
                </li>
                <li class="nav-item">
                    <a id="navAddCar" class="nav-link" href="/addCar">Add New
Car</a>
                </li>
                <li class="nav-item">
                    <a id="navLogout" class="nav-link"
href="/logout">LogOut</a>
                </li>
                <li class="nav-item">
                    <a id="navStaff" class="nav-link" href="/staff">Home</a>
                </li>
                <li class="nav-item">
                    <a id="navJob" class="nav-link" href="/job">My Job</a>
                </li>
                <li class="nav-item">
                    <a id="navAttendance" class="nav-link"
href="/attendance/{{ id }}">Attendance</a>
                </li>
                <li class="nav-item">
                    <a id="navJobHistory" class="nav-link"
href="/jobHistory">Job History</a>
                </li>
                <li class="nav-item">
                    <a id="navManager" class="nav-link"
href="/manager">Records</a>
                </li>
                <li class="nav-item">
                    <a id="navCreateStaffAccount" class="nav-link"
href="/createStaffAccount">Create Staff</a>
                </li>
                {% endblock %}
            </ul>
        </div>
    </nav>

    <!-- Main content area -->
    <div class="content">
        {% block body %}
        {% endblock %}
    </div>

    <!-- Footer section with copyright information -->
    <footer class="mt-5">
        &copy; 2025 Apple Garage Website
    </footer>

    <!-- libraries used -->
```


©2025 Apple Garage Website

```
    <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
    <script
src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.5.4/dist/umd/popper.min.js"
></script>
    <script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"><
/script>
</body>
</html>
```


©2025 Apple Garage Website

Landing.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items for this page -->
<li class="nav-item">
  <a id="navHome" class="nav-link" href="/">Home</a>
</li>
<li class="nav-item">
  <a id="navLogin" class="nav-link active" href="/login">LogIn</a>
</li>
<li class="nav-item">
  <a id="navSignUp" class="nav-link" href="/signUp">SignUp</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<div class="container">
    <div class="text-center">
        <!-- Display the logo -->
        <img src="{{url_for('static', filename='images/logo.png')}}"
alt="Apple Garage Logo" style="width: 400px; margin-top: 50px;">
    </div>

    <!-- Describe Web Section -->
    <div class="services my-5 py-4 text-center" id="services"
style="background-color: #e8f5e9; border-radius: 10px;">
        <h2 class="mb-3" style="color: #5cba00;">Our Services</h2>
        <p>We offer a variety of car services including maintenance, repair,
and MOT testing.</p>
    </div>
    <div class="about my-5 py-4 text-center" id="about" style="background-
color: #f1f8e9; border-radius: 10px;">
        <h2 class="mb-3" style="color: #5cba00;">About Us</h2>
        <p>Our team of experienced mechanics are here to provide the best
service possible.</p>
    </div>
    <div class="contact my-5 py-4 text-center" id="contact" style="background-
color: #dcedc8; border-radius: 10px;">
        <h2 class="mb-3" style="color: #5cba00;">Contact Us</h2>
        <p>Feel free to reach out to us for any inquiries or to schedule an
appointment.</p>
        <p>Our number: 07849511222</p>
    </div>
</div>
{% endblock %}
```


©2025 Apple Garage Website

Login.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
  <a id="navHome" class="nav-link" href="/">Home</a>
</li>
<li class="nav-item">
  <a id="navLogin" class="nav-link active" href="/login">LogIn</a>
</li>
<li class="nav-item">
  <a id="navSignUp" class="nav-link" href="/signUp">SignUp</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<div class="container">
  <div class="row justify-content-center">
      <div class="col-md-6">
          {% if error %}
          <!-- Display error message if login fails -->
          <div class="alert alert-danger text-center">
             {{ error }}
          </div>
          {% endif %}
          <div class="card mt-5" style="border: 20px solid #5cba00; border-
radius: 8px;">
              <div class="card-body">
                  <h3 class="card-title text-center mb-4">Login</h3>
                  <form action="/authenticate" method="POST">
                      <div class="form-group">
                          <label for="username">Username:</label>
                          <input type="text" id="username" name="username"
class="form-control" placeholder="Enter your username" required>
                      </div>
                      <div class="form-group mt-3">
                          <label for="password">Password:</label>
                          <input type="password" id="password" name="password"
class="form-control" placeholder="Enter your password" required>
                      </div>
                      <div class="form-group mt-4">
                          <input class="btn btn-primary btn-block"
type="submit" value="Login">
                      </div>
                  </form>
              </div>
```


©2025 Apple Garage Website

```
            </div>
            <!-- Additional links for sign-up and password reset -->
            <div class="text-center mt-3">
                <a href="/signUp">Don't have an account? Sign up</a>
            </div>
            <div class="text-center mt-3">
                <a href="/resetPassword">Reset password</a>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

SignUp.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
  <a id="navHome" class="nav-link" href="/">Home</a>
</li>
<li class="nav-item">
  <a id="navLogin" class="nav-link" href="/login">LogIn</a>
</li>
<li class="nav-item">
  <a id="navSignUp" class="nav-link active" href="/signUp">Sign Up</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-6">
            {% if error %}
            <!-- Display error message if present -->
            <div class="alert alert-danger text-center">
                {{ error }}
            </div>
            {% endif %}
            <div class="card mt-5" style="border: 20px solid #5cba00; border-
radius: 8px;">
                <div class="card-body">
                    <h3 class="text-center mb-4">Create Your Account</h3>
                    <form action="/createAccount" method="POST">
                        <div class="form-group">
                            <label for="username">Username:</label>
```


©2025 Apple Garage Website

```
                            <input type="text" class="form-control"
id="username" name="username" placeholder="Enter username" required>
                        </div>
                        <div class="form-group mt-3">
                            <label for="password">Password:</label>
                            <input type="password" class="form-control"
id="password" name="password" placeholder="Enter password" required>
                        </div>
                        <!-- Submit button -->
                        <div class="d-grid gap-2 mt-4">
                            <button type="submit" class="btn btn-primary btn-
block">Create Account</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

ResetPassword.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
  <a id="navHome" class="nav-link" href="/">Home</a>
</li>
<li class="nav-item">
  <a id="navLogin" class="nav-link" href="/login">LogIn</a>
</li>
<li class="nav-item">
  <a id="navSignUp" class="nav-link active" href="/signUp">Sign Up</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-6">
            {% if error %}
            <!-- Display error alert if an error exists -->
            <div class="alert alert-danger text-center">
                {{ error }}
            </div>
            {% endif %}
```


©2025 Apple Garage Website

```
            <div class="card mt-5" style="border: 20px solid #5cba00; border-
radius: 8px;">
                <div class="card-body">
                    <h3 class="text-center mb-4">Reset Your Password</h3>
                    <form action="/changePassword" method="POST">
                        <div class="form-group">
                            <label for="username">Username:</label>
                            <input type="text" class="form-control"
id="username" name="username" placeholder="Enter username" required>
                        </div>
                        <div class="form-group mt-3">
                            <label for="password">Password:</label>
                            <input type="password" class="form-control"
id="password" name="password" placeholder="Enter password" required>
                        </div>
                        <div class="form-group mt-3">
                            <label for="password">New Password:</label>
                            <input type="password" class="form-control"
id="newPassword" name="newPassword" placeholder="Enter new password" required>
                        </div>
                        <!-- Submit button -->
                        <div class="d-grid gap-2 mt-4">
                            <button type="submit" class="btn btn-primary btn-
block">Reset Password</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```


©2025 Apple Garage Website

Customer.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navAddCar" class="nav-link" href="/addCar">Add New Car</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<!-- Inline styling for the page -->
<head>
    <style>
        body {
            background-color: #5cba00;
            font-family: Arial, sans-serif;
        }
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            margin-bottom: 20px;
        }
        .car-title {
            font-size: 1.75em;
            font-weight: bold;
            color: black;
            margin-bottom: 10px;
        }
        .car-details {
            display: flex;
            justify-content: flex-start;
            gap: 30px;
            margin-bottom: 15px;
            font-size: 1.1em;
            color: white;
        }
        .car-buttons {
            display: flex;
            justify-content: flex-end;
```


©2025 Apple Garage Website

```
            gap: 10px;
        }
        .car-buttons .btn {
            margin-left: 5px;
            background-color: #007bff;
            color: white;
            border: none;
            padding: 8px 16px;
            border-radius: 4px;
            transition: background-color 0.3s ease;
        }
        .car-buttons .btn:hover {
            background-color: #0056b3;
        }
        .container {
            max-width: 800px;
            margin: auto;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5">
        <h1>My Car</h1>
        {% for car in car %}
        <div class="custom-container">
            <div class="car-title">
                {{ car[0] }}
            </div>
            <div class="car-details">
                <span>Make: {{ car[1] }}</span>
                <span>Model: {{ car[2] }}</span>
            </div>
            <div class="car-buttons">
                <a href="/car_detail/{{ car[0] }}" class="btn btn-
secondary">View Detail Of The Car</a>
                <a href="/record/{{ car[0] }}" class="btn btn-secondary">View
Maintenance Record</a>
                <a href="/booking/{{ car[0] }}" class="btn btn-secondary">Book
Service</a>
                <a href="/delete_car/{{ car[0] }}" class="btn btn-
secondary">Delete This Car</a>
            </div>
        </div>
        {% endfor %}
    </div>
</body>
{% endblock %}
```


©2025 Apple Garage Website

AddCar.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navAddCar" class="nav-link" href="/addCar">Add New Car</a>
</li>
{% endblock %}

{% block body %}
<!-- Inline style for custom container -->
<head>
    <style>
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }
    </style>
</head>
<!-- Main content area -->
<div class="container mt-5 custom-container">
    <div class="card">
        <div class="card-body">
            <h3 class="card-title text-center">Enter Registration Number</h3>
            {% if error %}
            <!-- Display error message if exists -->
            <div class="alert alert-danger text-center">
                {{ error }}
            </div>
            {% endif %}
            <form action="/regNumber" method="POST">
                <div class="form-group">
                    <label for="regNumber">Registration Number:</label>
                    <input type="text" name="regNumber" id="regNumber"
class="form-control" required>
                </div>
                <div class="mt-4 d-flex justify-content-between">
                    <a href="/customer" class="btn btn-secondary">Back</a>
                    <input class="btn btn-primary" type="submit"
value="Submit">
                </div>
            </form>
        </div>
```


©2025 Apple Garage Website

```
    </div>
</div>
{% endblock %}
```

Error.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navAddCar" class="nav-link" href="/addCar">Add New Car</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<head>
    <title>Vehicle Details</title>
</head>
<body>
    <div class="container mt-5">
        <div class="row justify-content-center">
            <div class="col-md-6">
                <div class="alert alert-danger text-center" role="alert">
                    <strong>Error!</strong> Can't find this car.
                </div>
                <div class="text-center mt-4">
                    <a href="/addCar" class="btn btn-secondary">Back</a>
                </div>
            </div>
        </div>
    </div>
    <script>
        document.querySelector('#navHome').classList.add("active");
    </script>
</body>
{% endblock %}
```


©2025 Apple Garage Website

Detail.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navAddCar" class="nav-link" href="/addCar">Add New Car</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Styling for the custom container */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5 custom-container">
        <h1 class="mt-5">Vehicle Details</h1>
        <ul class="list-group mt-3">
            {% for i in range(num) %}
                <!-- Loop over each vehicle detail -->
                <li class="list-group-item d-flex justify-content-between
align-items-center">
                    <strong>{{ title[i] }}:</strong>
                    <span>{{ detail[i] }}</span>
                </li>
            {% endfor %}
        </ul>
        {% if title[-1] != "model" %}
            <div class="card mt-4">
                <div class="card-body">
                    <form method="post">
                        <div class="form-group">
                            <strong for="model">Enter Car Model:</strong>
                            <input type="text" class="form-control" id="model"
name="model" required>
                        </div>
```


©2025 Apple Garage Website

```
                        <button type="submit" class="btn btn-
primary">Submit</button>
                    </form>
                </div>
            </div>
        {% endif %}
        <div class="mt-4 d-flex justify-content-between">
            <a href="/addCar" class="btn btn-secondary">Back</a>
            {% if title[-1] == "model" %}
            <form action="/store_detail" method="POST">
                <button type="submit" class="btn btn-primary">Save
detail</button>
            </form>
            {% endif %}
        </div>
    </div>
</body>
{% endblock %}
```


©2025 Apple Garage Website

Car_detail.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- No navbar items defined in this template -->
{% endblock %}

{% block body %}
<head>
    <style>
        /* Define custom styling for the container */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5 custom-container">
        <h1 class="mt-5">Vehicle Details</h1>
        <ul class="list-group mt-3">
            {% for i in range(num) %}
                <!-- Loop through each vehicle detail -->
                <li class="list-group-item d-flex justify-content-between
align-items-center">
                    <strong>{{ title[i] }}:</strong>
                    <span>{{ detail[i] }}</span>
                </li>
            {% endfor %}
            <div class="mt-4 d-flex justify-content-between">
                <a id="backButton" href="/customer" class="btn btn-
secondary">Back</a>
            </div>
        </ul>
    </div>
</body>
<script>
    // Navback based on user type
    function setBackButtonHref() {
        var backButton = document.getElementById('backButton');
        var referrer = document.referrer;
        if (referrer.includes('/staff')) {
            backButton.href = '/staff';
        } else {
            backButton.href = '/customer';
        }
```


©2025 Apple Garage Website

```
    }
    window.onload = setBackButtonHref;
</script>
{% endblock %}
```

Booking.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }

        .custom-container h2 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }

        .form-group label {
            font-weight: 500;
            color: black;
        }

        .form-control {
            border-radius: 5px;
            padding: 10px;
            border: 1px solid black;
        }

        .d-flex {
            display: flex;
```


©2025 Apple Garage Website

```
            justify-content: space-between;
            align-items: center;
        }
    </style>
</head>
<!-- Main content area -->
<div class="container mt-5 custom-container">
    <div class="card">
        <div class="card-body">
            <h2>Book Your Service</h2>
            <form action="/add_booking/{{reg}}" method="POST">
                <div class="form-group">
                    <label for="service">Select Service:</label>
                    <select class="form-control" id="service" name="service"
required>
                        <option value="">-- Please choose an option --
</option>
                        <option value="MOT">MOT</option>
                        <option value="Oil Change">Oil Change</option>
                        <option value="Full Service">Full Service (Oil Change,
Air Filter Change, Fuel Filter Change, Pollen Filter Change)</option>
                        <option value="Brakes">Brakes</option>
                        <option value="Engine Parts">Engine Parts</option>
                        <option value="Suspension & Steering">Suspension &
Steering</option>
                        <option value="Transmission">Transmission</option>
                        <option value="Cooling & Heating">Cooling &
Heating</option>
                        <option value="Electrics & Lighting">Electrics &
Lighting</option>
                        <option value="Other">Other</option>
                    </select>
                </div>
                <!-- Additional input for 'Other' service, hidden by default -
->
                <div class="form-group" id="otherServiceContainer"
style="display: none;">
                    <label for="otherService">Please specify:</label>
                    <input type="text" class="form-control" id="otherService"
name="otherService" placeholder="Specify your service">
                </div>
                <div class="form-group" id="detail">
                    <label for="otherService">Additional
Notes(Optional):</label>
                    <input type="text" class="form-control" id="detail"
name="detail" placeholder="-- Write Down Your Additional Notes --">
                </div>
                <div class="form-group">
```


©2025 Apple Garage Website

```
                    <label for="date">Select Date:</label>
                    <input type="date" class="form-control" id="date"
name="date" value="{{ request.form.get('date', '') }}" min="{{ today }}"
required>
                </div>
                <div class="mt-4 d-flex justify-content-between">
                    <a href="/customer" class="btn btn-secondary">Back</a>
                    <button type="submit" class="btn btn-primary">Book
Now</button>
                </div>
            </form>
        </div>
    </div>
</div>

<script>
    // Determine the visibility of the 'Other' service input based on
selection
    document.getElementById('service').addEventListener('change', function() {
        var otherServiceContainer =
document.getElementById('otherServiceContainer');
        if (this.value === 'Other') {
            otherServiceContainer.style.display = 'block';
        } else {
            otherServiceContainer.style.display = 'none';
        }
    });
</script>
{% endblock %}
```


©2025 Apple Garage Website

Record.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navCustomer" class="nav-link" href="/customer">My Car</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }

        h1 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }

        .row-header {
            background-color: gray;
            color: white;
            padding: 10px;
            border-radius: 5px;
            font-weight: 600;
        }

        .row-data {
            background-color: white;
            padding: 10px;
            border-bottom: 1px solid gray;
            transition: background-color 0.3s ease;
        }

        .row-data .col {
            display: flex;
            align-items: center;
```


©2025 Apple Garage Website

```
            justify-content: center;
            text-align: center;
        }

        a.cancel-link {
            color: red;
            text-decoration: none;
            font-weight: 600;
        }

        a.cancel-link:hover {
            text-decoration: underline;
        }

        .d-flex {
            display: flex;
            justify-content: flex-start;
            align-items: center;
            margin-top: 20px;
        }
    </style>
</head>
<!-- Main content area -->
<div class="container mt-5 custom-container">
    <div class="card">
        <div class="card-body">
            <h1>Maintenance Record</h1>
            <!-- Header row for service record columns -->
            <div class="row row-header text-center">
                <div class="col">Maintenance Items</div>
                <div class="col">Current Status</div>
                <div class="col">Maintenance Date</div>
                <div class="col">Total Expenditure</div>
                <div class="col">Cancel Scheduled Booking</div>
            </div>
            {% for record in record %}
            <div class="row row-data text-center">
                <div class="col">{{ record[0] }}</div>
                <div class="col">{{ record[1] }}</div>
                <div class="col">{{ record[2] }}</div>
                <div class="col">
                    {% if record[3] != None %}
                        £{{ record[3] }}
                    {% else %}
                        {{ record[3] }}
                    {% endif %}
                </div>
                <div class="col">
```


©2025 Apple Garage Website

```
                    {% if record[1] == "Scheduled" %}
                    <!-- Show cancel link only for scheduled records -->
                    <a href="/delete_record/{{ record[5] }}/{{ record[4] }}"
class="cancel-link">Cancel</a>
                    {% endif %}
                </div>
            </div>
            {% endfor %}
            <!-- Back button -->
            <div class="d-flex">
                <a href="/customer" class="btn btn-secondary">Back</a>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

Staff.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navStaff" class="nav-link" href="/staff">Home</a>
</li>
<li class="nav-item">
    <a id="navAttendance" class="nav-link"
href="/attendance/{{ id }}">Attendance</a>
</li>
<li class="nav-item">
    <a id="navJobHistory" class="nav-link" href="/jobHistory">Job History</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        body {
            background-color: #5cba00;
            font-family: Arial, sans-serif;
        }
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
```


©2025 Apple Garage Website

```
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            margin-bottom: 20px;
        }
        .job-title {
            font-size: 1.75em;
            font-weight: bold;
            color: black;
            margin-bottom: 10px;
        }

        .job-details {
            display: flex;
            justify-content: flex-start;
            gap: 30px;
            margin-bottom: 15px;
            font-size: 1.1em;
            color: white;
        }

        .job-buttons {
            display: flex;
            justify-content: flex-end;
            gap: 10px;
        }

        .job-buttons .btn {
            margin-left: 5px;
            background-color: #007bff;
            color: white;
            border: none;
            padding: 8px 16px;
            border-radius: 4px;
            transition: background-color 0.3s ease;
        }
        .job-buttons .btn:hover {
            background-color: #0056b3;
        }

        .container {
            max-width: 800px;
            margin: auto;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5">
```


©2025 Apple Garage Website

```
        <h1>Available Jobs</h1>
        {% if error %}
        <!-- Display error message if present -->
        <div class="alert alert-danger text-center">
            {{ error }}
        </div>
        <a href="/staff" class="btn btn-secondary">Refresh</a>
        {% endif %}
        {% for job in job %}
        <!-- Loop over each available job -->
        <div class="custom-container">
            <div class="job-title">
                {{ job[1] }}
            </div>
            <div class="job-details">
                <span>Registration: {{ job[0] }}</span>
                <span>Make: {{ job[2] }}</span>
                <span>Model: {{ job[3] }}</span>
                <span>Status: {{ job[5] }}</span>
            </div>
            <div class="job-buttons">
                <!-- Action buttons for the job -->
                <a href="/car_detail/{{ job[0] }}" class="btn btn-
secondary">View Detail Of This Car</a>
                <a href="/additional_notes/{{ job[4] }}" class="btn btn-
secondary">View Additional Notes By Customer</a>
                <a
href="/store_staffId/{{ job[0] }}/{{ job[4] }}/{{ job[2] }}" class="btn btn-
secondary">Starting this job</a>
            </div>
        </div>
        {% endfor %}
    </div>
</body>
{% endblock %}
```


©2025 Apple Garage Website

Additional_note.html

```
{% extends 'layout.html' %}
{% block navbar %}
<!-- No navbar content defined for this page -->
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom container styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5 custom-container">
        <h1 class="mt-5">Additional Notes</h1>
        <div class="card shadow-sm">
            <div class="card-body">
                <div class="job-details">
                    <p class="text-muted">{{ detail }}</p>
                </div>
            </div>
            <div class="card-footer text-end">
                <a id="backButton" href="/staff" class="btn btn-
secondary">Back</a>
            </div>
        </div>
    </div>
</body>
<script>
    // Set the back button's href based on the referrer URL
    function setBackButtonHref() {
        var backButton = document.getElementById('backButton');
        var referrer = document.referrer;
        if (referrer.includes('/staff')) {
            backButton.href = '/staff';
        } else {
            backButton.href = '/job';
        }
    }
    window.onload = setBackButtonHref;
</script>
{% endblock %}
```


©2025 Apple Garage Website

Job.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navJob" class="nav-link" href="/job">My Job</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
            color: black;
        }
        .card {
            background-color: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            margin-bottom: 20px;
        }
        .form-group label {
            font-weight: bold;
        }
        .d-grid a, .d-grid button {
            width: 100%;
            margin-bottom: 10px;
        }
        h1 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }
        h4 {
            color: black;
        }
        .row-header {
            background-color: gray;
            color: white;
            padding: 10px;
            border-radius: 5px;
```


©2025 Apple Garage Website

```
        font-weight: 600;
    }
    .row-data {
        background-color: white;
        padding: 10px;
        border-bottom: 1px solid gray;
        transition: background-color 0.3s ease;
        word-wrap: break-word;
        word-break: break-word;
    }
    .row-data .col {
        display: flex;
        align-items: center;
        justify-content: center;
        text-align: center;
    }
    .d-flex {
        display: flex;
        justify-content: flex-start;
        align-items: center;
        margin-top: 20px;
    }
    .button-group {
        display: flex;
        flex-wrap: wrap;
        gap: 15px;
        justify-content: center;
        margin-bottom: 30px;
    }
    .button-group a {
        flex: 1 1 100%;
        max-width: 300px;
        text-align: center;
    }
    .back-to-top {
        position: fixed;
        bottom: 20px;
        right: 20px;
        display: none;
        background-color: #007bff;
        color: white;
        border: none;
        padding: 10px 15px;
        border-radius: 50%;
        font-size: 18px;
        cursor: pointer;
        z-index: 1000;
    }
```


©2025 Apple Garage Website

```
        .back-to-top:hover {
            background-color: #0056b3;
        }
        [id] {
            scroll-margin-top: 80px;
        }
    </style>
</head>
<body>
    <!-- Main content area -->
    <div class="container mt-5 custom-container">
        <!-- Navigation jump buttons -->
        <div class="button-group">
            <a href="#add-staff" class="btn btn-primary">Jump to Add Staff</a>
            <a href="#car-detail" class="btn btn-primary">Jump to Car
Detail</a>
            <a href="#car-parts" class="btn btn-primary">Jump to Car Parts</a>
            <a href="#car-problem" class="btn btn-primary">Jump to Car
Problem</a>
            <a href="#additional-notes" class="btn btn-primary">Jump to
Additional Notes By Customer</a>
            <a href="#job-status" class="btn btn-primary">Jump to Manage Job
Status</a>
        </div>
        <h1 class="text-center">Working on {{ type }} - {{ reg }}</h1>
        <!-- Add Staff Section -->
        <div id="add-staff" class="card">
            <h4 class="text-center mb-3">Add Staff You Are Working With</h4>
            <form action="/addStaff" method="POST">
                <div class="form-group mb-3">
                    {% with error =
get_flashed_messages(with_categories=True) %}
                        {% if error %}
                            {% for category, error in error %}
                                <div class="alert alert-danger text-center">
                                    {{ error }}
                                </div>
                            {% endfor %}
                        {% endif %}
                    {% endwith %}
                    <label for="staff">Staff Name:</label>
                    <select class="form-control" id="staff" name="staff"
required>
                        <option value="">-- Please choose a staff --</option>
                        {% for each in staff %}
                            <option value="{{ each }}">{{ each }}</option>
                        {% endfor %}
                    </select>
```


©2025 Apple Garage Website

```
                </div>
                <div class="d-grid gap-2 mt-4">
                    <button type="submit" class="btn btn-primary btn-
block">Add</button>
                </div>
            </form>
        </div>
        <!-- Car Detail Section -->
        <div id="car-detail" class="card">
            <div class="card-body">
                <h1>{{ reg }}:Car Detail</h1>
                <div class="row row-header text-center">
                    <div class="col">Title</div>
                    <div class="col">Detail</div>
                </div>
                {% for i in range(num) %}
                <div class="row row-data text-center">
                    <div class="col">{{ title[i] }}:</div>
                    <div class="col">{{ detail[i] }}</div>
                </div>
                {% endfor %}
            </div>
        </div>
        <!-- Additional Notes Section -->
        <div id="additional-notes" class="card shadow-sm">
            <h1 class="mt-5">Additional Notes By Customer</h1>
            <div class="card-body">
                <div class="job-details">
                    <p class="text-muted">{{ note }}</p>
                </div>
            </div>
        </div>
        <!-- Car Parts Section -->
        <div id="car-parts" class="card">
            <h4 class="text-center mb-3">Add Car Parts Used</h4>
            <form action="/addParts" method="POST">
                <div class="form-group mb-3">
                    <label for="parts">Car Parts:</label>
                    <input type="text" class="form-control" id="parts"
name="parts" placeholder="Enter Part Name" required>
                </div>
                <div class="form-group mb-3">
                    <label for="parts">Euro Car Parts product code:</label>
                    <input type="text" class="form-control" id="partId"
name="partId" placeholder="Enter Product code" required>
                </div>
                <div class="d-grid gap-2 mt-4">
```


©2025 Apple Garage Website

```
                    <button type="submit" class="btn btn-primary btn-
block">Add Car Parts</button>
                </div>
            </form>
        </div>
        <!-- Car Parts Used History Section -->
        <div class="card">
            <div class="card-body">
                <h1>{{ reg }}:Car Parts Used History</h1>
                <div class="row row-header text-center">
                    <div class="col">Part Name</div>
                    <div class="col">Euro Car Parts product code</div>
                    <div class="col">Service Item</div>
                    <div class="col">Date</div>
                </div>
                {% for parts in parts %}
                <div class="row row-data text-center">
                    <div class="col">{{ parts[0] }}</div>
                    <div class="col">{{ parts[3] }}</div>
                    <div class="col">{{ parts[1] }}</div>
                    <div class="col">{{ parts[2] }}</div>
                </div>
                {% endfor %}
            </div>
        </div>
        <!-- Car Problem Section -->
        <div id="car-problem" class="card">
            <h4 class="text-center mb-3">Add Problem Encountered</h4>
            <form action="/addProblem" method="POST">
                <div class="form-group mb-3">
                    <label for="problem">Enter Problem:</label>
                    <input type="text" class="form-control" id="problem"
name="problem" placeholder="Enter Problem" required>
                </div>
                <div class="d-grid gap-2 mt-4">
                    <button type="submit" class="btn btn-primary btn-
block">Add Problem</button>
                </div>
            </form>
        </div>
        <!-- Problem History Section -->
        <div class="card">
            <div class="card-body">
                <h1>{{ make }},{{ model }}:Problem History</h1>
                <div class="row row-header text-center">
                    <div class="col">Problem</div>
                    <div class="col">Service Item</div>
                    <div class="col">Date</div>
```


©2025 Apple Garage Website

```
                </div>
                {% for problem in problem %}
                <div class="row row-data text-center">
                    <div class="col">{{ problem[0] }}</div>
                    <div class="col">{{ problem[1] }}</div>
                    <div class="col">{{ problem[2] }}</div>
                </div>
                {% endfor %}
            </div>
        </div>
        <!-- Manage Job Status Section -->
        <div id="job-status" class="card">
            <h4 class="text-center mb-3">Manage Job Status</h4>
            <div class="d-grid gap-2">
                <a href="/pause/{{ serviceId }}" class="btn btn-warning">Pause
Job</a>
                <a href="/complete/{{ serviceId }}" class="btn btn-
success">Complete Job</a>
            </div>
        </div>
        <!-- Back-to-top button -->
        <button id="backToTop" class="back-to-top">↑</button>
    </div>
    <script>
        // Display the back-to-top button after scrolling down
        window.onscroll = function() {
            var backToTop = document.getElementById('backToTop');
            if (document.body.scrollTop > 100 ||
document.documentElement.scrollTop > 100) {
                backToTop.style.display = "block";
            } else {
                backToTop.style.display = "none";
            }
        };
        document.getElementById('backToTop').onclick = function() {
            window.scrollTo({top: 0, behavior: 'smooth'});
        };
    </script>
</body>
{% endblock %}
```


©2025 Apple Garage Website

Attendance.html

```
{% extends 'layout.html' %}
{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navStaff" class="nav-link" href="/staff">Home</a>
</li>
<li class="nav-item">
    <a id="navAttendance" class="nav-link"
href="/attendance/{{ id }}">Attendance</a>
</li>
<li class="nav-item">
    <a id="navJobHistory" class="nav-link" href="/jobHistory">Job History</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}
{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }

        h2 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }

        table {
            width: 100%;
            border-collapse: collapse;
            margin-bottom: 20px;
        }

        th, td {
            border: 1px solid white;
            text-align: center;
            padding: 8px;
        }

        th {
```


©2025 Apple Garage Website

```
            background-color: gray;
            color: white;
            padding: 10px;
            font-weight: 600;
        }

        .green {
            background-color: green;
            color: white;
        }
        .red {
            background-color: red;
            color: white;
        }

        .d-flex {
            display: flex;
            justify-content: flex-start;
            align-items: center;
            margin-top: 20px;
        }
    </style>
</head>
<!-- Main content area -->
<div class="container mt-5 custom-container">
    <div class="card">
        <div class="card-body">
            <h2>Attendance for Last 30 Days</h2>
            <table>
                <thead>
                    <tr>
                        <th>Date</th>
                        <th>Attendance</th>
                    </tr>
                </thead>
                <tbody>
                    {% for date, color in date_color %}
                    <tr>
                        <td>{{ date }}</td>
                        <td class="{{ color }}"></td>
                    </tr>
                    {% endfor %}
                </tbody>
            </table>
        </div>
    </div>
</div>
{% endblock %}
```


©2025 Apple Garage Website

jobHistory.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navStaff" class="nav-link" href="/staff">Home</a>
</li>
<li class="nav-item">
    <a id="navAttendance" class="nav-link"
href="/attendance/{{ id }}">Attendance</a>
</li>
<li class="nav-item">
    <a id="navJobHistory" class="nav-link" href="/jobHistory">Job History</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }

        h1 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }

        .row-header {
            background-color: gray;
            color: white;
            padding: 10px;
            border-radius: 5px;
            font-weight: 600;
        }

        .row-data {
            background-color: white;
            padding: 10px;
```


©2025 Apple Garage Website

```
            border-bottom: 1px solid gray;
            transition: background-color 0.3s ease;
        }

        .row-data .col {
            display: flex;
            align-items: center;
            justify-content: center;
            text-align: center;
        }

        .d-flex {
            display: flex;
            justify-content: flex-start;
            align-items: center;
            margin-top: 20px;
        }
    </style>
</head>
<!-- Main content area -->
<div class="container mt-5 custom-container">
    <div class="card">
        <div class="card-body">
            <h1>My Job History</h1>
            <div class="row row-header text-center">
                <div class="col">Maintenance Items</div>
                <div class="col">Current Status</div>
                <div class="col">Maintenance Date</div>
                <div class="col">Car Make</div>
                <div class="col">Car Model</div>
                <div class="col">Registration Number</div>
            </div>
            {% for record in record %}
            <div class="row row-data text-center">
                <div class="col">{{ record[0] }}</div>
                <div class="col">{{ record[1] }}</div>
                <div class="col">{{ record[2] }}</div>
                <div class="col">{{ record[3] }}</div>
                <div class="col">{{ record[4] }}</div>
                <div class="col">{{ record[5] }}</div>
            </div>
            {% endfor %}
        </div>
    </div>
</div>
{% endblock %}
```


©2025 Apple Garage Website

Manager.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navManager" class="nav-link" href="/manager">Records</a>
</li>
<li class="nav-item">
    <a id="navCreateStaffAccount" class="nav-link"
href="/createStaffAccount">Create Staff</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        h1 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }
        .table-container {
            width: 90%;
            margin: 0 auto;
            overflow-y: auto;
            max-height: 70vh;
            border: 1px solid black;
        }
        table {
            width: 100%;
            border-collapse: collapse;
        }
        table, th, td {
            border: 1px solid black;
        }
        th, td {
            padding: 12px;
            text-align: center;
        }
        th {
            background-color: #5cba00;
            color: white;
```


©2025 Apple Garage Website

```
            font-weight: 600;
            position: sticky;
            top: 0;
            z-index: 2;
        }
        td a {
            display: inline-block;
            padding: 5px 10px;
            color: white;
            border-radius: 4px;
            text-decoration: none;
        }
        td .cancel-link {
            color: red;
            text-decoration: none;
        }
        td .cancel-link:hover {
            text-decoration: underline;
        }
        td input[type="text"] {
            width: 100%;
            padding: 5px;
            box-sizing: border-box;
        }
        td button {
            width: 100%;
            padding: 5px;
            border: none;
            color: white;
            border-radius: 4px;
            cursor: pointer;
        }
    </style>
</head>
<!-- Main content area -->
<h1>Service Record</h1>
<div class="table-container">
    <table>
        <thead>
            <tr>
                <th>RecordId</th>
                <th>Make</th>
                <th>Model</th>
                <th>Maintenance Item</th>
                <th>Status</th>
                <th>Date</th>
                <th>Leader Staff</th>
                <th>Assist Staff</th>
```


©2025 Apple Garage Website

```
                <th>Total Expenditure</th>
                <th>Detail</th>
                <th>Cancel Scheduled Booking</th>
                <th>Enter Total Charge</th>
            </tr>
        </thead>
        <tbody>
            {% for record, color in record_color %}
            <!-- Each row is styled with a background color based on status -
->
            <tr style="background-color: {{ color }};">
                <td data-label="serviceId">{{ record[4] }}</td>
                <td data-label="Make">{{ record[7] }}</td>
                <td data-label="Model">{{ record[8] }}</td>
                <td data-label="Maintenance Items">{{ record[0] }}</td>
                <td data-label="Current Status">{{ record[1] }}</td>
                <td data-label="Maintenance Date">{{ record[2] }}</td>
                <td data-label="Leader Staff">{{ record[5] }}</td>
                <td data-label="Help Staff">{{ record[6] }}</td>
                <td data-label="Total Spend">
                    {% if record[3] != None %}
                        £{{ record[3] }}
                    {% else %}
                        {{ record[3] }}
                    {% endif %}
                </td>
                <td data-label="Service Detail">
                    <!-- Link to view service detail -->
                    <a href="/serviceDetail/{{ record[4] }}" class="btn btn-
secondary">View</a>
                </td>
                <td data-label="Cancel">
                    <!-- Show cancel link if status is "Scheduled" -->
                    {% if record[1] == "Scheduled" %}
                        <a href="/deleteRecord/{{ record[4] }}" class="cancel-
link">Cancel</a>
                    {% endif %}
                </td>
                <td data-label="Price">
                    <!-- If the job is done, allow entering the total charge -
->
                    {% if record[1] == "Done" %}
                    <form action="/enterPrice/{{ record[4] }}" method="POST">
                        <input type="text" class="form-control" id="price"
name="price" placeholder="Enter Charge (£)">
                        <button type="submit" class="btn btn-
primary">Submit</button>
                    </form>
```


©2025 Apple Garage Website

```
                    {% endif %}
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endblock %}
```

serviceDetail.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!--Custom navbar items -->
<li class="nav-item">
  <a id="navManager" class="nav-link" href="/manager">Services</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<head>
    <style>
        /* Custom page styling */
        .custom-container {
            background-color: #5cba00;
            padding: 20px;
            border-radius: 8px;
        }
        h1 {
            color: black;
            margin-bottom: 20px;
            font-weight: bold;
            text-align: center;
        }
        .row-header {
            background-color: gray;
            color: white;
            padding: 10px;
            border-radius: 5px;
            font-weight: 600;
        }
        .row-data {
            background-color: white;
            padding: 10px;
            border-bottom: 1px solid gray;
```


©2025 Apple Garage Website

```
            transition: background-color 0.3s ease;
        }
        .row-data .col {
            display: flex;
            align-items: center;
            justify-content: center;
            text-align: center;
        }
        .d-flex {
            display: flex;
            justify-content: flex-start;
            align-items: center;
            margin-top: 20px;
        }
    </style>
</head>
<div class="container mt-5 custom-container">
    <!-- Main content area -->
    <div class="card">
        <!-- Problem Section -->
        <div class="card-body">
            <h1>Problems</h1>
            <div class="row row-header text-center">
                <div class="col">No.</div>
                <div class="col">Problem</div>
            </div>
            {% for problems in problems %}
            <div class="row row-data text-center">
                <div class="col">{{ loop.index }}</div>
                <div class="col">{{ problems[0] }}</div>
            </div>
            {% endfor %}
        </div>
    </div>
    <!-- Car Parts Used Section -->
    <div class="card">
        <div class="card-body">
            <h1>Car Parts Used</h1>
            <div class="row row-header text-center">
                <div class="col">No.</div>
                <div class="col">Parts Name</div>
            </div>
            {% for parts in parts %}
            <div class="row row-data text-center">
                <div class="col">{{ loop.index }}</div>
                <div class="col">{{ parts[0] }}</div>
            </div>
            {% endfor %}
```


©2025 Apple Garage Website

```
            <div class="d-flex">
                <a href="/manager" class="btn btn-secondary">Back</a>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```

CreateStaffAccount.html

```
{% extends 'layout.html' %}

{% block navbar %}
<!-- Custom navbar items -->
<li class="nav-item">
    <a id="navManager" class="nav-link" href="/manager">Records</a>
</li>
<li class="nav-item">
    <a id="navCreateStaffAccount" class="nav-link"
href="/createStaffAccount">Create Staff</a>
</li>
<li class="nav-item">
    <a id="navLogout" class="nav-link" href="/logout">LogOut</a>
</li>
{% endblock %}

{% block body %}
<!-- Main content area -->
<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-6">
            {% if error %}
            <!-- Display error message if present -->
            <div class="alert alert-danger text-center">
                {{ error }}
            </div>
            {% endif %}
            <div class="card mt-5" style="border: 20px solid #5cba00; border-
radius: 8px;">
                <div class="card-body">
                    <h3 class="text-center mb-4">Create Staff Account</h3>
                    <form action="/create_Staff_Account" method="POST">
                        <div class="form-group">
                            <label for="username">Username:</label>
                            <input type="text" class="form-control"
id="username" name="username" placeholder="Enter username" required>
                        </div>
                        <div class="form-group mt-3">
                            <label for="password">Password:</label>
```


©2025 Apple Garage Website

```
                            <input type="password" class="form-control"
id="password" name="password" placeholder="Enter password" required>
                        </div>
                        <div class="d-grid gap-2 mt-4">
                            <button type="submit" class="btn btn-primary btn-
block">Create Account</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
</div>
{% endblock %}
```


©2025 Apple Garage Website

4. Testing

Objectives number	description	Test result	pass
1.1, 1.2	Landing page and login system	Watch video: https://youtu.be/Wrs92FiJd_s	
1.3, 1.4	Sign up system and reset password	Watch video: https://youtu.be/9dHWOfAiKXE	
2.1, 2.6	Customer page and add new car	Watch video: https://youtu.be/e0NWrDL1sPg https://youtu.be/atqBCLRyrGI	
2.2, 2.3, 2.4, 2.5	Customer page functions and booking new service	Watch video: https://youtu.be/2jJqkJRmnGU	
3.1	Staff landing page	Watch video: https://youtu.be/YsUIwa_Nuos	
3.2	Working page	Watch video: https://youtu.be/OflZ78VxKMw	
3.3, 3.4	Attendance page and job history page	Watch video: https://youtu.be/iv0vtheygus	
4.1, 4.1, 4.3	Manager page and functions	Watch video: https://youtu.be/NFyMt9YId50 https://youtu.be/jXZZKC1h8UI	
4.4	Create staff account	Watch video: https://youtu.be/8iOeo4zVNOk	
5.1, 5.2	Nav bar	The nav bar was displayed and being used in each of previous test videos.	

©2025 Apple Garage Website

5. Evaluation

5.1. Objectives

Objectives number	description	comment	met
1.1.1	The logo	Displayed on landing page. Also, as small icon on the navbar	
1.1.2	The detail and explanation of the garage	Displayed on landing page.	
1.2.1	Allow users to enter their username and password to login	Implemented input box for user to enter username and password	
1.2.2	Have button to jump to sign up page	Displayed on login page.	
1.2.3	Have button to jump to reset password page	Displayed on login page.	
1.2.4	Redirect to correct user page based on user type once login successfully		
1.3.1	Allow users to enter their username and password to create account	Implemented input box for user to enter username and password	
1.3.2	Redirect to login page once create account successfully		
1.4.1	let users to enter correct old password and then replace it with new password	Implemented input box for user to enter their old password and new password	
2.1.1	list vehicles owned with registration number, make and model	Displayed on customer landing page	
2.1.2	Customer able view details of the car	User able to view by click button on customer landing page	
2.1.3	Customer able to view maintenance records	User able to view by click button on customer landing page	
2.1.4	Book service button jumps to book service page	Displayed on customer landing page	
2.1.5	delete car button	Displayed on customer landing page	
2.2	Vehicle detail page, list of vehicle information from API		
2.3.1	Maintenance record page, display to the user all maintenance records on their car including maintenance item, status, date and total expenditure		

©2025 Apple Garage Website

2.3.2	Allow user to cancel their booking if maintenance status is scheduled	Implemented cancel button for any scheduled booking	
2.4.1	Book service page, allow user to select service, date and add additional note if need.		
2.5	Delete car, removing all the record on the vehicle	Implemented delete button on customer landing page	
2.6	Add new car page, let user add their car to the system by entering registration number		
2.7	logout	Implemented logout button on nav bar.	
3.1	Staff landing page, list of available jobs, including maintenance item, registration number, make, model and status		
3.1.1	staff can start any scheduled jobs	Implemented start button on each of available job	
3.1.2	staff can restart any paused job which paused by themselves	Implemented restart button on each of paused job	
3.2	working page, page for staff during the maintenance period		
3.2.1	staff can add staff they are working together with	Implemented input box to add staff working with.	
3.2.2	staff can view details of the vehicle they are working with	Displayed on working page	
3.2.3	staff can view additional notes by customer	Displayed on working page	
3.2.4	staff can add car parts they used during the service by enter its name and product code	Implemented input box to add car part used.	
3.2.5	staff can view car parts used history of the vehicle they are working with	Displayed on working page	
3.2.6	staff can add problem they encountered during the service	Implemented input box to add problem encountered	
3.2.7	staff can view problem history of the vehicle model they are working with	Displayed on working page	
3.2.8	staff can pause job	Implemented pause job button on working page	
3.2.9	staff can complete job	Implemented complete job button on working page	

©2025 Apple Garage Website

3.3	Attendance Page, staff can see their attendance for last 30 days.	Displayed as table red represents absent and green represents present	
3.4	Job history page, staff can see the table of jobs they did which including maintenance items, current status, maintenance date, car make, car model and registration number.		
3.5	logout	Implemented logout button on nav bar.	
4.1.1	service record table, manager can see all jobs did by staffs. Table includes recordId, car make, car model, maintenance item, status, maintenance date, leader staff, assist staff and total expenditure.		
4.1.2	record in table should list in order from new date to old.		
4.1.3	use different colours to represent records in different status in the table.		
4.2	manager able to cancel scheduled bookings	Implemented cancel button next to any scheduled bookings	
4.3	manager able to enter total charge for service when its status is Done	Implemented input box to enter total charge for any service was done.	
4.4	Create staff account page, allow user to enter username and password to create staff account	Implemented input box for user to create username and password	
4.5	logout	Implemented logout button on nav bar.	
5.1	nav bar will on top of every page with different options based on the user type		
5.1.1	if user haven't login nav bar will display home, login and signup		
5.1.2	if user type is customer nav bar will display my car, add new car and logout		
5.1.3	if user type is staff nav bar will display home, attendance, job history and logout		
5.1.4	if user type is manager nav bar will display records, create staff, logout		

©2025 Apple Garage Website

5.2	nav bar will also display username if user logged in		

Conclusion

I have sufficiently fulfilled all the outlined objectives established in my analysis.

©2025 Apple Garage Website

5.2. User feedback

I asked my father's boss who is the owner of the garage my father works. I let him use my website as different types of users. Here's his feedback:

- **As customer**: Signing up and logging in is straightforward. When I create an account and then log in, it immediately taken to my customer page, which lists my cars. The "Add Car" process is interesting because it calls the external API to fetch vehicle details. Booking a service is simple—you select a service type, choose a date, and add any notes. However, a confirmation message or indicator after booking would help me to make sure that my booking went through.

- **As staff**: The automatic check-in on the day I log in is a nice touch. A clear notification after check-in would be better to confirm that my attendance has been recorded. The staff page lists available jobs with details like the car's registration number, make/model, and service status. The ability to start, pause, or complete a job is useful.

- **As manager**: The manager page provides a table of service records with color-coding for different statuses (e.g., light blue for Scheduled, green for Done). This visual cue is very useful for quickly view overall progress. Adding filters or search options could help manage large numbers of records.

©2025 Apple Garage Website

5.3. Improvements

Based on user feedback I find 3 area which I could add into my solution to make it more practical and user friendly.

1. The first improvement I could make is add message or indicator when user booking their service successfully. This will help customers to understand they complete their booking without to look at maintenance record page. I could also add message reminder on the day user booked their service to make sure user doesn't forgot their booking.

2. The second improvement I could make is add notification when staff check in. If this could implement, staff will no longer need to turn on the attendance page to see whether their check in is successful or not.

3. The third improvement I could make is add search functionality on manager page. Allowing manager to search records could save quite an amount of time when records numbers are large. I could also add filters for example: only display one particularly staff's service record or only display particularly date's record. This could help manager tracking the information they want without tracking the whole record table.

©2025 Apple Garage Website

Appendix - Techniques used

skill	description	Page reference
Data Models and Structures	**Complex relational databases with multiple interlinked tables. Many to many relationships (user – service)**	**Pg. 53**
Client-Server and Web Technologies	**Server-side scripting using request and response objects with server-side extensions**	**Pg. 58**
Client-Server and Web Technologies	**User authentication system with hashed passwords**	**Pg. 58**
Client-Server and Web Technologies	**Calling parameterised web service APIs and parsing JSON to service a complex client-server model**	**Pg. 62**
Data Models and Structures	**Cross-table parameterised SQL queries**	**Pg. 69**
Client-Server and Web Technologies	**Complex client-server models. Allows multiple users using the web on the same time.**	**Pg. 115 (especially test video for object 3.2)**

©2025 Apple Garage Website